

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Разработка программно-информационной системы для организации

и проведения занятий йогой

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

В. Е. Бабичев

(инициалы, фамилия)

Группа ПО-216

Руководитель ВКР

(подпись, дата)

Т. Н. Конаныхина

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

«_____» _____ 20____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Бабичева В. Е., шифр 22-06-0207, группа ПО-216

1. Тема «Разработка программно-информационной системы для организации и проведения занятий йогой» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1584-с.

2. Срок предоставления работы к защите «09» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) провести анализ предметной области йога-студий и существующих решений для записи на занятия;

2) разработать концептуальную модель мобильного приложения «Йога-студия», определить основные сущности и связи между ними;

3) спроектировать архитектуру приложения (Single Activity + Fragments) и пользовательский интерфейс;

4) разработать и протестировать мобильное приложение для платформы Android на языке Kotlin с использованием SharedPreferences.

3.2. Входные данные и требуемые результаты для программы:

1) **Входными данными** для программной системы являются: регистрационные данные пользователей (имя, email, пароль), профильные дан-

ные пользователей (аватар, город, уровень подготовки), данные справочников занятий и инструкторов, параметры записи на занятия, текстовые сообщения отзывов и оценки, поисковые запросы и параметры фильтрации по городу.

2) **Выходными данными** для программной системы являются: сформированный каталог занятий с фильтрацией по городу, расписание с календарём, профиль пользователя с аватаром и настройками, список записей пользователя, отзывы и рейтинг занятий, статистика прогресса в виде графиков, уведомления о подтверждении записи и напоминания о занятиях.

4. Содержание работы (по разделам):

4.1. Введение.

4.2. Анализ предметной области.

4.3. Техническое задание: основание для разработки, назначение разработки, требования к программной системе, требования к оформлению документации.

4.4. Технический проект: общие сведения о программной системе, проект данных программной системы, проектирование архитектуры программной системы, проектирование пользовательского интерфейса программной системы.

4.5. Рабочий проект: спецификация компонентов и классов программной системы, тестирование программной системы.

4.6. Заключение.

4.7. Список использованных источников.

4.8. Приложения (листинги программного кода, плакаты).

5. Перечень графического материала:

Лист 1. Сведения о ВКР.

Лист 2. Цель и задачи разработки.

Лист 3. Архитектурная схема приложения.

Лист 4. Диаграмма классов.

Лист 5. ER-диаграмма сущностей.

Лист 6. Диаграмма прецедентов.

Лист 7. Заключение.

Руководитель ВКР

(подпись, дата)

Т. Н. Конаныхина

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

В. Е. Бабичев

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 100 страницам. Работа содержит 27 иллюстраций, 6 таблиц, 27 библиографических источников и 7 листов графического материала.

Ключевые слова: мобильное приложение; йога-студия; Android; Kotlin; запись на занятия; расписание; профиль пользователя; уведомления; отзывы; статистика; графики; SharedPreferences; Material Design.

Объект разработки: мобильное приложение «Йога-студия» для платформы Android, предназначенное для организации и проведения занятий йогой, автоматизации записи клиентов, управления расписанием и отслеживания прогресса.

Цель работы: разработка программно-информационной системы для организации и проведения занятий йогой.

Методы и средства: при разработке использовались язык программирования Kotlin, средства разработки Android SDK, локальное хранилище SharedPreferences, библиотеки MPAndroidChart для построения графиков и CircleImageView для отображения аватаров пользователей. Применялись методы объектно-ориентированного анализа и проектирования, язык UML для моделирования.

Результаты: в процессе создания приложения были реализованы модуль авторизации и регистрации пользователей, каталог занятий с фильтрацией по городу, расписание с календарём, профиль пользователя, система отзывов и рейтинга, статистика и графики прогресса, а также уведомления и напоминания.

Область применения: разработанное приложение может использоваться йога-студиями и фитнес-центрами для автоматизации записи клиентов, а также отдельными пользователями для отслеживания личного прогресса.

СОДЕРЖАНИЕ

РЕФЕРАТ	5
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	9
ВВЕДЕНИЕ	11
1 Анализ предметной области	13
1.1 Особенности работы йога-студий и их деятельности	13
1.2 Проблематика традиционного подхода к управлению записями	13
1.3 Обоснование необходимости автоматизации процессов	14
1.4 Структура предметной области и ключевые сущности	14
1.5 Анализ существующих решений в тематике записи на занятия	16
1.6 Анализ целевой аудитории	22
1.7 Выводы по разделу	23
2 Техническое задание	24
2.1 Основание для разработки	24
2.2 Цель и назначение разработки	24
2.3 Требования к функциональности	24
2.4 Нефункциональные требования	25
2.5 Требования к интерфейсу	26
2.6 Моделирование вариантов использования	26
2.7 Требования к надёжности	33
2.8 Требования к информационной безопасности	34
2.9 Требования к программной и технической совместимости	34
2.10 Спецификация функциональных требований	34
2.11 Требования к оформлению документации	36
3 Технический проект	37
3.1 Архитектура программной системы	37
3.2 Обоснование выбора архитектуры Single Activity	37
3.3 Обоснование выбора технологий проектирования	39
3.4 Клиентская часть	41
3.5 Диаграмма компонентов	42

3.6	Диаграмма классов	42
3.7	Диаграмма последовательности	44
3.8	Диаграмма состояний	44
3.9	Диаграмма взаимодействия фрагментов	46
3.10	Диаграмма пакетов	46
3.11	Диаграмма развертывания	48
3.12	Проект базы данных (SharedPreferences)	48
3.13	Надёжность, производительность и масштабирование	50
3.14	Информационная безопасность технического проекта	51
3.15	Эксплуатация и развёртывание	51
3.16	Верификация технического проекта	52
3.17	Выводы по разделу	53
4	Рабочий проект	54
4.1	Реализация модуля авторизации и регистрации	54
4.2	Реализация каталога занятий	55
4.3	Реализация записи на занятие	55
4.4	Реализация расписания с календарём	56
4.5	Реализация профиля пользователя	57
4.6	Реализация системы отзывов и рейтинга	58
4.7	Реализация статистики и графиков	59
4.8	Реализация уведомлений и напоминаний	60
4.9	Детальное описание алгоритмов работы модулей	60
4.10	Интерфейс приложения	63
4.11	Тестирование приложения	66
5	Тестирование программной системы	67
5.1	Цели и задачи тестирования	67
5.2	План тестирования	67
5.3	Функциональное тестирование	67
5.4	Тестирование пользовательского интерфейса	70
5.5	Тестирование производительности	71
5.6	Тестирование в условиях ограниченных ресурсов	72

5.7 Совместимость с версиями Android	72
5.8 Выявленные дефекты и их устранение	72
5.9 Выводы по тестированию	73
ЗАКЛЮЧЕНИЕ	74
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	75
ПРИЛОЖЕНИЕ А Представление графического материала	79
ПРИЛОЖЕНИЕ Б Листинги программного кода	87
В.1 Модуль авторизации (AuthActivity.kt)	87
В.2 Модуль каталога занятий (ClassesFragment.kt)	88
В.3 Модуль профиля пользователя (ProfileFragment.kt)	90
В.4 Модуль расписания (ScheduleFragment.kt)	91
В.5 Модуль отзывов и рейтинга (ReviewFragment.kt)	93
В.6 Модуль статистики и графиков (ProgressFragment.kt)	94
В.7 Модуль уведомлений (NotificationHelper.kt)	96
В.8 Модели данных (Data Models.kt)	98
В.9 SharedPreferences Manager (PrefsManager.kt)	99
Сведения о ВКР (Графический материал / Сведения о ВКР.png)	Лист 1
Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 2
Архитектурная схема приложения (Графический материал / Архитектурная схема приложения.png)	Лист 3
Диаграмма классов (Графический материал / Диаграмма классов.png)	Лист 4
ER-диаграмма сущностей (Графический материал / ER-диаграмма сущностей.png)	Лист 5
Диаграмма прецедентов (Графический материал / Диаграмма прецедентов.png)	Лист 6
Заключение (Графический материал / Заключение.png)	Лист 7

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

Русские сокращения

- **АПК** – аппаратно-программный комплекс.
- **БД** – база данных.
- **ИС** – информационная система.
- **ИТ** – информационные технологии.
- **ПО** – программное обеспечение.
- **ПС** – программная система.
- **РП** – рабочий проект.
- **ТЗ** – техническое задание.
- **ТП** – технический проект.
- **UI** – пользовательский интерфейс (User Interface).
- **UX** – пользовательский опыт (User Experience).

Английские сокращения

- **API** (Application Programming Interface) – интерфейс программирования приложений.
- **CSV** (Comma-Separated Values) – текстовый формат для представления табличных данных.
- **IDE** (Integrated Development Environment) – среда разработки.
- **JSON** (JavaScript Object Notation) – текстовый формат обмена данными.
- **JVM** (Java Virtual Machine) – виртуальная машина Java.
- **SDK** (Software Development Kit) – набор средств разработки.
- **SQL** (Structured Query Language) – язык структурированных запросов.
- **XML** (eXtensible Markup Language) – расширяемый язык разметки.

Специальные термины (используемые в работе)

- **Activity** – компонент Android, представляющий один экран с пользовательским интерфейсом.

- **Android SDK** – набор инструментов для разработки приложений под Android.
- **BottomNavigationView** – компонент навигации с нижним меню в Android.
- **Fragment** – компонент Android, представляющий часть интерфейса внутри Activity.
- **Kotlin** – современный язык программирования для Android.
- **RecyclerView** – компонент Android для отображения прокручиваемых списков.
- **SharedPreferences** – механизм Android для локального хранения пар «ключ-значение».
- **ViewPager2** – компонент Android для создания прокручиваемых экранов с вкладками.

ВВЕДЕНИЕ

В современном мире йога как практика физического и духовного развития становится всё более популярной. Миллионы людей занимаются йогой для поддержания здоровья, снятия стресса и улучшения качества жизни. Однако многие йога-студии сталкиваются с проблемами автоматизации процесса записи клиентов на занятия, управления расписанием и отслеживания прогресса.

Традиционные методы управления (запись по телефону, журналы, таблицы Excel) являются неэффективными и отнимают много времени как у администраторов, так и у клиентов. Клиенты хотят иметь возможность записаться на занятие в любое время, получить напоминание и отслеживать свой прогресс. Администраторам необходим инструмент для управления расписанием, инструкторами и просмотра всех записей.

Актуальность данной работы обусловлена необходимостью создания удобного, бесплатного и автономного инструмента для записи на занятия йогой, который мог бы использоваться как отдельными студиями, так и конечными пользователями.

Объектом исследования являются процессы автоматизации записи на занятия в йога-студиях.

Предметом исследования — методы и средства разработки мобильных приложений для организации занятий йогой.

Целью данной работы является разработка программно-информационной системы для организации и проведения занятий йогой в виде мобильного приложения для платформы Android.

Для достижения поставленной цели необходимо решить следующие задачи:

1. провести анализ предметной области и существующих аналогов;
2. сформулировать функциональные и нефункциональные требования к разрабатываемой системе;
3. обосновать выбор технологий разработки;

4. разработать архитектуру приложения;
5. спроектировать пользовательский интерфейс;
6. реализовать основные модули приложения;
7. выполнить тестирование разработанного приложения.

Методы исследования: анализ научной и технической литературы, сравнительный анализ существующих решений, моделирование с использованием UML, объектно-ориентированное проектирование, тестирование программного обеспечения.

Научная новизна заключается в разработке автономного мобильного приложения для записи на занятия йогой, не требующего подключения к интернету и использования серверной инфраструктуры.

Практическая значимость работы состоит в том, что разработанное приложение может быть использовано йога-студиями и фитнес-центрами для автоматизации записи клиентов, а также отдельными пользователями для отслеживания личного прогресса.

Результатом работы является мобильное приложение «Йога-студия» для платформы Android, позволяющее пользователям просматривать расписание, записываться на занятия, отслеживать прогресс, оставлять отзывы и получать уведомления.

1 Анализ предметной области

1.1 Особенности работы йога-студий и их деятельности

Йога как практика физического и духовного развития существует тысячи лет. В современном мире йога стала массовым явлением: миллионы людей занимаются ею для поддержания здоровья, снятия стресса и улучшения качества жизни. Йога-студии предлагают различные направления, включая хатха-йогу, виньтасу, аштангу, кундалини, инь-йогу и медитацию.

Современные йога-студии сталкиваются с рядом проблем. В первую очередь, это необходимость управления расписанием занятий и обработка записей клиентов. Также требуется информирование клиентов об изменениях в расписании, отслеживание прогресса клиентов и сбор обратной связи в виде отзывов и рейтингов. Кроме того, важной задачей является управление информацией об инструкторах.

1.2 Проблематика традиционного подхода к управлению записями

Традиционные методы управления, такие как запись по телефону, ведение журналов или таблиц Excel, являются неэффективными и отнимают много времени как у администраторов, так и у клиентов. При отсутствии специализированной системы данные о клиентах и записях распределены по разным источникам, что приводит к ряду типовых проблем.

Разрозненность данных проявляется в том, что одни и те же записи дублируются в нескольких местах, создавая расхождения в информации о клиентах и занятиях. Сложность актуализации заключается в том, что любое изменение расписания требует ручной правки в нескольких каналах, из-за чего часть информации быстро устаревает. Высокие издержки пользовательского поиска вынуждают клиента переключаться между источниками, чтобы получить полную картину о занятиях. Отсутствие единого профиля пользователя не позволяет отслеживать историю посещений и прогресс. Кроме того, кли-

енты не имеют возможности получать своевременные напоминания о предстоящих занятиях.

1.3 Обоснование необходимости автоматизации процессов

Автоматизация в рассматриваемой предметной области необходима не сама по себе, а как инструмент устранения перечисленных противоречий. Для йога-студии автоматизация обеспечивает единое информационное пространство, в котором данные о клиентах, расписании, записях и отзывах поддерживаются согласованно.

Практический эффект автоматизации проявляется в нескольких аспектах. Сокращается время обновления расписания и обработки записей клиентов. Снижается вероятность ошибок при повторном ручном вводе данных. Повышается доступность информации для конечного пользователя. Упрощается отслеживание прогресса клиентов. Формируется единая логика управления профилями и записями. Таким образом, автоматизация является необходимым условием устойчивой работы сервиса, ориентированного на регулярное использование и постоянное развитие.

1.4 Структура предметной области и ключевые сущности

На основании анализа деятельности сервиса выделяются следующие ключевые сущности. Пользователь представляет собой клиента йога-студии, имеющего профиль с личными данными, уровнем подготовки и историей записей. Занятие является основной единицей, включающей название, описание, время проведения, инструктора, город и максимальное количество участников. Инструктор — это специалист, проводящий занятия, с информацией об имени, специализации и городе работы. Запись представляет собой связь между пользователем и занятием, фиксирующую намерение клиента посетить занятие. Отзыв содержит оценку и комментарий пользователя о посещённом занятии. Прогресс отражает статистику посещений пользователя за определённый период.

1.4.1 Детальное описание сущности «Пользователь»

Пользователь является центральной сущностью системы. Каждый пользователь имеет уникальный идентификатор, адрес электронной почты (используемый для входа), пароль (хранится в зашифрованном виде), имя, отображаемое в профиле, город проживания (влияет на фильтрацию занятий), уровень подготовки (новичок, средний, продвинутый), а также аватар (опционально). Пользователь может просматривать каталог занятий, записываться на них, отменять запись, оставлять отзывы и отслеживать свой прогресс.

1.4.2 Детальное описание сущности «Занятие»

Занятие представляет собой основную единицу, предоставляемую йога-студией. Каждое занятие характеризуется следующими атрибутами: уникальный идентификатор, название (например, «Хатха-йога для начинающих»), подробное описание, день и время проведения, продолжительность, инструктор, город проведения, максимальное количество участников, текущее количество записанных, цена (если занятие платное), а также флаг публикации (видимо для пользователей или скрыто). Занятия группируются по направлениям и городам.

1.4.3 Детальное описание сущности «Инструктор»

Инструктор — это специалист, проводящий занятия. Каждый инструктор имеет уникальный идентификатор, имя, специализацию (направления йоги, которые он преподаёт), город работы, краткую биографию, фотографию и рейтинг (рассчитываемый на основе отзывов пользователей). Инструктор может вести несколько занятий в разных городах. Пользователи могут просматривать профиль инструктора и оставлять отзывы о его занятиях.

1.4.4 Детальное описание сущности «Запись»

Запись фиксирует намерение пользователя посетить конкретное занятие. Запись содержит идентификатор пользователя, идентификатор занятия, дату и время создания записи, статус (активна, отменена, посещена). Система проверяет наличие свободных мест перед созданием записи. При отмене записи место освобождается и становится доступным для других пользователей. После проведения занятия статус записи меняется на «посещено», и пользователь получает возможность оставить отзыв.

1.4.5 Детальное описание сущности «Отзыв»

Отзыв позволяет пользователю оценить качество проведённого занятия. Отзыв содержит идентификатор пользователя, идентификатор занятия, рейтинг (от 1 до 5 звёзд), текстовый комментарий, дату создания. Пользователь может оставить отзыв только после фактического посещения занятия (статус записи «посещено»). Отзывы влияют на рейтинг инструктора и помогают другим пользователям при выборе занятий.

1.4.6 Детальное описание сущности «Прогресс»

Прогресс отражает статистику посещений пользователя за определённый период. Система автоматически обновляет прогресс после каждого посещённого занятия. Прогресс включает общее количество занятий, количество занятий по месяцам, количество занятий по направлениям йоги, а также динамику изменения уровня подготовки. Данные о прогрессе визуализируются в виде графиков с использованием библиотеки MPAndroidChart.

1.5 Анализ существующих решений в тематике записи на занятия

На рынке представлено несколько типов решений для записи на занятия йогой. В таблице 1.1 приведён сравнительный анализ существующих аналогов.

Таблица 1.1 – Сравнительный анализ существующих решений

Критерий	TimePad	FitStars	Excel/ручной	Наше решение
Бесплатно	Нет	Нет	Да	Да
Мобильное приложение	Нет	Да	Нет	Да
Запись онлайн	Да	Да	Нет	Да
Профиль пользователя	Нет	Да	Нет	Да
Отслеживание прогресса	Нет	Частично	Нет	Да
Отзывы и рейтинг	Нет	Да	Нет	Да
Уведомления	Нет	Да	Нет	Да
Офлайн-режим	Нет	Нет	Да	Да

1.5.1 Зарубежные платформы

На международном рынке существует ряд решений для управления фитнес-студиями. Платформа Mindbody является одним из лидеров, предоставляя комплексные возможности для управления расписанием, записями и платежами. Однако данный сервис является платным и ориентирован преимущественно на крупные сети студий. Сервис Zen Planner предлагает аналогичный функционал, но также требует ежемесячной платы и не имеет полноценной поддержки русского языка. Ключевым недостатком зарубежных решений является их ориентация на западный рынок и отсутствие бесплатных тарифов для малых студий.

Платформа Mindbody — это комплексное решение для управления фитнес-бизнесом, включающее модули для расписания, записи клиентов, обработки платежей, маркетинга и аналитики. Система интегрируется с различными платёжными шлюзами и предлагает мобильное приложение для клиентов. Однако стоимость подписки начинается от 129 долларов в месяц, что недоступно для малых йога-студий.

Платформа ClassPass — это не столько инструмент управления студией, сколько агрегатор занятий, позволяющий пользователям покупать абоне-

менты и записываться на занятия в различных студиях. С точки зрения студии, ClassPass привлекает новых клиентов, но студия теряет контроль над ценообразованием и взаимоотношениями с клиентами.

1.5.2 Региональные и отечественные решения

В Российской Федерации также представлены решения для управления занятиями. FitStars является мобильным приложением, предлагающим каталог занятий и запись онлайн, однако его функционал ограничен, а некоторые возможности требуют платной подписки. TimePad представляет собой платформу для управления мероприятиями, которая может использоваться для записи на занятия, но не имеет мобильного приложения и ориентирована скорее на разовые события, чем на регулярные занятия. Многие студии продолжают использовать ручную запись через Excel и мессенджеры, что подтверждает актуальность разработки нового решения.

Платформа FitStars предлагает бесплатную версию с ограниченным функционалом (до 50 записей в месяц). Платная подписка (от 990 рублей в месяц) открывает доступ к неограниченному количеству записей, статистике и интеграции с социальными сетями. Однако приложение не предоставляет возможности отслеживания прогресса клиентов и не имеет офлайн-режима.

Платформа TimePad изначально создавалась для управления мероприятиями (конференции, семинары, тренинги). Она поддерживает создание событий, продажу билетов и рассылку уведомлений. Однако интерфейс не оптимизирован для регулярных занятий (нет понятия «абонемент», «прогресс», «инструктор»), а мобильное приложение отсутствует.

1.5.3 Выводы по анализу существующих решений

Проведённый анализ позволяет выделить несколько ключевых принципов, на которых строятся успешные современные приложения для записи на занятия. Пользователи ожидают возможность записываться на занятия со смартфона в любое время, поэтому мобильная платформа является обязательным требованием. Приложение должно сохранять базовую функцио-

нальность даже без подключения к интернету, обеспечивая автономную работу. Наличие профиля пользователя, отслеживание прогресса и персональные рекомендации повышают вовлечённость. Возможность оставлять отзывы и оценки формирует доверие к сервису. Напоминания о занятиях снижают количество пропусков и повышают удовлетворённость клиентов.

Существующие решения, такие как TimePad и FitStars, либо не поддерживают мобильные устройства, либо являются платными, либо не предоставляют полного набора функций для клиентов йога-студий. Таким образом, разработка собственного мобильного приложения «Йога-студия» является актуальной и целесообразной.

1.5.4 Детальное сравнение с аналогами

Для более глубокого анализа существующих решений проведём сравнение по расширенному набору критериев. В таблице 1.2 представлено детальное сравнение платформ TimePad, FitStars, ClassPass, Mindbody и разрабатываемого приложения «Йога-студия».

Таблица 1.2 – Детальное сравнение существующих решений

Критерий	TimePad	FitStars	ClassPass	Mindbody	Наше решение
Бесплатно	Нет	Нет	Нет	Нет	Да
Мобильное приложение	Нет	Да	Да	Да	Да
Офлайн-режим	Нет	Нет	Нет	Нет	Да
Запись он-лайн	Да	Да	Да	Да	Да

Таблица 1.2 – Детальное сравнение существующих решений (продолжение)

Критерий	TimePad	FitStars	ClassPass	Mindbody	Наше решение
Отмена записи	Да	Да	Да	Да	Да
Профиль пользователя	Нет	Да	Да	Да	Да
История записей	Частично	Да	Да	Да	Да
Отслеживание прогресса	Нет	Частично	Нет	Да	Да
Отзывы и рейтинг	Нет	Да	Да	Да	Да
Уведомления	Нет	Да	Да	Да	Да
Напоминания	Нет	Да	Да	Да	Да
Календарь занятий	Да	Да	Да	Да	Да
Фильтрация по городу	Да	Да	Да	Да	Да
Список инструкторов	Частично	Да	Да	Да	Да
Статистика и графики	Нет	Нет	Нет	Да	Да
Русский язык интерфейса	Да	Да	Нет	Нет	Да

Таблица 1.2 – Детальное сравнение существующих решений (продолжение)

Критерий	TimePad	FitStars	ClassPass	Mindbody	Наше решение
Техническая поддержка	Платная	Платная	Платная	Платная	Бесплатная
Адаптация под малый бизнес	Нет	Частично	Нет	Нет	Да

Анализ таблицы позволяет сделать следующие выводы.

TimePad является удобной платформой для управления мероприятиями, но не имеет мобильного приложения и не поддерживает офлайн-режим. Отсутствуют функции отслеживания прогресса, отзывов и статистики. Данное решение подходит для разовых событий, но не для регулярных занятий йогой.

FitStars предоставляет мобильное приложение с базовым набором функций: запись онлайн, отмена записи, отзывы и уведомления. Однако отсутствует офлайн-режим, а функция отслеживания прогресса реализована лишь частично. Бесплатная версия имеет ограничения по количеству записей.

ClassPass является агрегатором занятий, позволяющим пользователям записываться в различные студии. С точки зрения отдельной йога-студии, это инструмент привлечения клиентов, но не управления. Студия теряет контроль над ценообразованием и взаимоотношениями с клиентами.

Mindbody представляет собой наиболее функциональное решение среди аналогов, предоставляющее полный набор функций: запись, прогресс, отзывы, статистику. Однако оно является платным, не имеет офлайн-режима и русскоязычного интерфейса, а также ориентировано преимущественно на крупные сети студий.

Разрабатываемое приложение «Йога-студия» сочетает в себе все необходимые функции: является бесплатным, работает в офлайн-режиме, имеет русскоязычный интерфейс, адаптировано под малый бизнес. Приложение предоставляет полный цикл записи на занятия, отслеживание прогресса, систему отзывов и рейтинга, а также уведомления и напоминания.

1.6 Анализ целевой аудитории

Целевая аудитория разрабатываемого приложения может быть разделена на две основные группы: клиенты йога-студий и администраторы (включая инструкторов).

1.6.1 Портрет клиента йога-студии

Клиент йога-студии — это человек в возрасте от 18 до 55 лет, ведущий активный образ жизни, интересующийся здоровьем и саморазвитием. Основные потребности клиента: возможность быстро записаться на занятие, получать напоминания, отслеживать свой прогресс, оставлять отзывы. Клиент предпочитает использовать мобильное приложение, так как это удобно и быстро. Большинство клиентов используют смартфоны на базе Android (около 70% рынка).

1.6.2 Портрет администратора йога-студии

Администратор йога-студии — это сотрудник студии, отвечающий за управление расписанием, обработку записей и взаимодействие с клиентами. Основные потребности администратора: возможность быстро изменять расписание, просматривать список записанных клиентов, отслеживать загрузженность залов и инструкторов, получать отчёты.

1.6.3 Портрет инструктора

Инструктор — это специалист, проводящий занятия. Основные потребности инструктора: просмотр расписания своих занятий, просмотр списка

записанных клиентов, возможность отмечать посещаемость, отслеживание своего рейтинга на основе отзывов.

1.7 Выводы по разделу

На основе проведённого анализа было принято решение разработать мобильное приложение «Йога-студия» на платформе Android с использованием языка Kotlin. Приложение будет включать полный цикл записи на занятия, профиль пользователя, статистику, отзывы и уведомления. Выбранная архитектура обеспечивает модульность, удобство поддержки и масштабирования.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра «Разработка программно-информационной системы для организации и проведения занятий йогой», утверждённое приказом ректора ЮЗГУ от «16» апреля 2026 г. №1859-с. Разработка ведётся в соответствии с требованиями ГОСТ 34.602-89 «Техническое задание на создание автоматизированной системы».

2.2 Цель и назначение разработки

Программный продукт предназначен для автоматизации процесса записи клиентов на занятия йогой, управления расписанием и отслеживания прогресса пользователей. Целевой аудиторией приложения являются клиенты йога-студий, которые получают удобный инструмент для просмотра расписания, записи на занятия и отслеживания прогресса, а также администраторы и инструкторы, которые используют приложение для управления расписанием, просмотра записей и анализа загруженности.

2.3 Требования к функциональности

В таблице 2.1 приведён перечень функциональных требований к разрабатываемому приложению.

Таблица 2.1 – Функциональные требования

Идентификатор	Описание требования
FR-01	Авторизация пользователя с проверкой учётных данных
FR-02	Регистрация нового пользователя (имя, email, пароль, уровень подготовки)
FR-03	Просмотр каталога занятий с фильтрацией по городу
FR-04	Просмотр детальной информации о занятии
FR-05	Запись на выбранное занятие
FR-06	Отмена существующей записи
FR-07	Просмотр расписания с календарём на выбранную дату
FR-08	Редактирование профиля пользователя (аватар, имя, email, город, уровень)
FR-09	Просмотр списка инструкторов с фильтрацией по городу
FR-10	Оставление отзыва и рейтинга (от 1 до 5 звёзд)
FR-11	Просмотр статистики прогресса в виде графиков
FR-12	Просмотр списка активных записей пользователя
FR-13	Получение уведомления о записи на занятие
FR-14	Получение напоминания о занятии за 1 час

2.4 Нефункциональные требования

В таблице 2.2 приведены нефункциональные требования к разрабатываемому приложению.

Таблица 2.2 – Нефункциональные требования

Категория	Описание требования
Совместимость	Минимальная версия Android 7.0 (API level 24)
Локализация	Интерфейс на русском языке
Автономность	Работа без подключения к интернету (локальное хранение данных)
Производительность	Время отклика интерфейса не более 1 секунды
Надёжность	Сохранение данных пользователя при перезапуске приложения
Удобство использования	Интуитивно понятная навигация
Безопасность	Пароли хранятся в зашифрованном виде
Масштабируемость	Возможность добавления новых типов занятий без изменения кода

2.5 Требования к интерфейсу

Приложение должно включать в себя следующие экраны и элементы: навигацию по разделам с помощью BottomNavigationView с пятью вкладками, экран авторизации и регистрации, главный экран с приветствием и рекомендациями, каталог занятий с фильтрацией по городу, экран детальной информации о занятии, расписание с календарём, экран инструкторов с фильтрацией по городу, профиль пользователя с аватаром и настройками, экран статистики с графиками, экран моих записей, а также экран отзывов.

Интерфейс должен быть выполнен в едином стиле с использованием компонентов Material Design, обеспечивающих интуитивно понятное взаимодействие. Все элементы управления должны быть подписаны на русском языке. Шрифты должны быть достаточно крупными для комфортного чтения на экранах с диагональю от 5 дюймов.

2.6 Моделирование вариантов использования

На рисунке 2.1 представлена диаграмма прецедентов, демонстрирующая функциональные возможности приложения для пользователя.

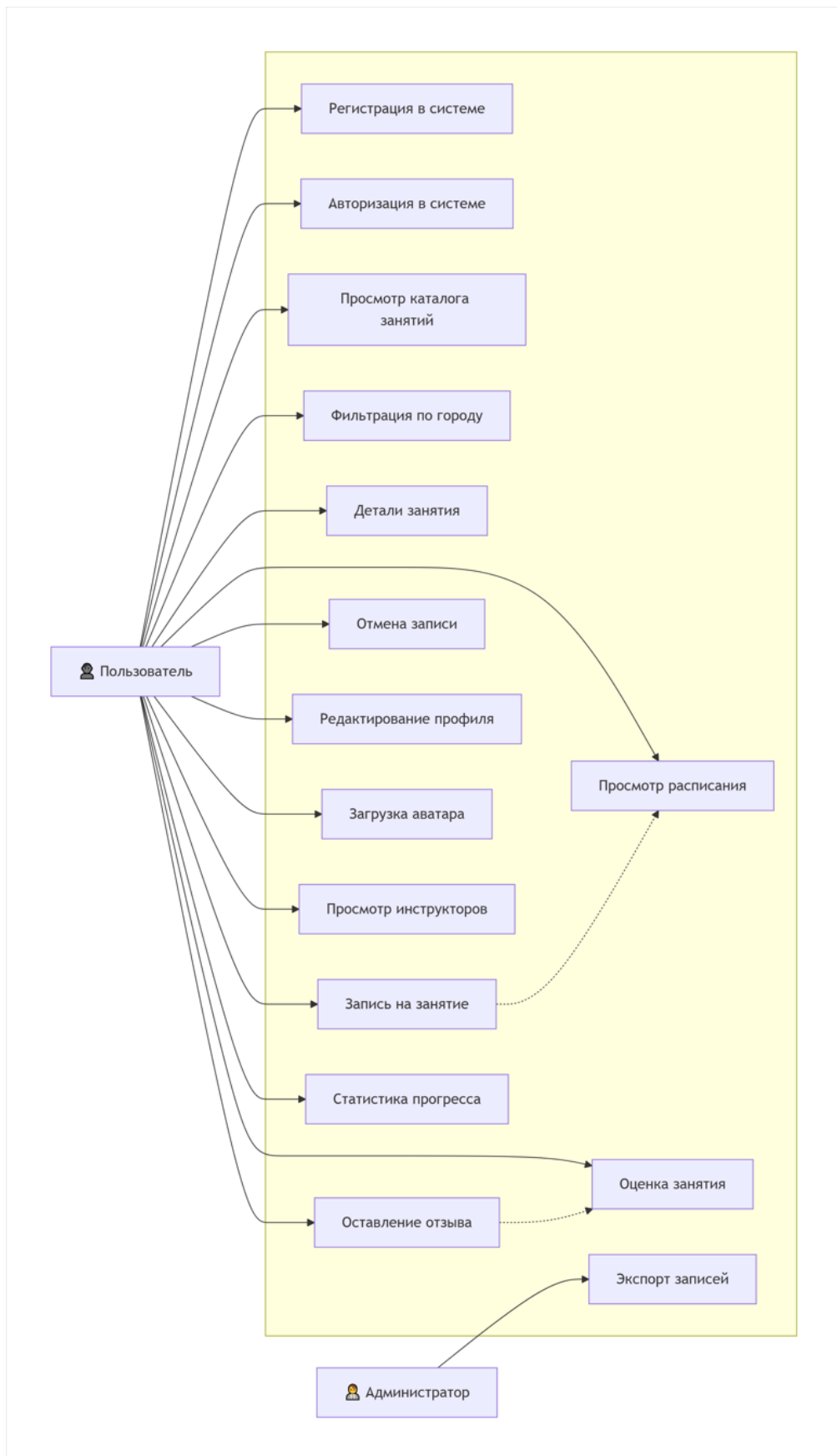


Рисунок 2.1 – Диаграмма прецедентов

Далее приведены агрегированные перечни прецедентов для каждой роли; они детализируются в последующих подпунктах.

2.6.1 Вариант использования «Регистрация пользователя»

Предусловие: приложение открыто на экране авторизации, пользователь не авторизован.

Постусловие: создана новая учётная запись, пользователь авторизован и перенаправлен на главный экран.

Основной сценарий:

1. Пользователь нажимает кнопку «Регистрация».
2. Система отображает форму регистрации.
3. Пользователь вводит имя, адрес электронной почты, пароль и уровень подготовки.
4. Пользователь нажимает кнопку «Зарегистрироваться».
5. Система проверяет, не зарегистрирован ли уже указанный email.
6. Если email свободен, система сохраняет данные в SharedPreferences.
7. Система автоматически выполняет вход и открывает главный экран.

Альтернативный сценарий A1 (email уже зарегистрирован):

1. Система обнаруживает, что указанный email уже существует.
2. Система отображает сообщение «Пользователь с таким email уже существует».
3. Регистрация не выполняется, форма остаётся открытой.

Альтернативный сценарий A2 (не заполнены обязательные поля):

1. Система обнаруживает, что не все обязательные поля заполнены.
2. Система подсвечивает незаполненные поля и отображает сообщение «Заполните все поля».
3. Регистрация не выполняется.

2.6.2 Вариант использования «Авторизация пользователя»

Предусловие: приложение открыто на экране авторизации, пользователь зарегистрирован.

Постусловие: пользователь авторизован и перенаправлен на главный экран.

Основной сценарий:

1. Пользователь вводит адрес электронной почты и пароль.
2. Пользователь нажимает кнопку «Войти».
3. Система проверяет соответствие данных в SharedPreferences.
4. Данные верны, система открывает главный экран.

Альтернативный сценарий А1 (неверный email или пароль):

1. Система проверяет данные и обнаруживает несоответствие.
2. Система отображает сообщение «Неверный email или пароль».
3. Пользователь остаётся на экране авторизации.

2.6.3 Вариант использования «Просмотр каталога занятий»

Предусловие: пользователь авторизован, открыт главный экран.

Постусловие: отображён список доступных занятий.

Основной сценарий:

1. Пользователь нажимает на вкладку «Занятия».
2. Система загружает список занятий из SharedPreferences.
3. Система отображает карточки занятий на экране.
4. Пользователь просматривает список, прокручивая его.

Альтернативный сценарий А1 (список занятий пуст):

1. Система обнаруживает, что список занятий пуст.
2. Система отображает сообщение «Нет доступных занятий».

2.6.4 Вариант использования «Фильтрация занятий по городу»

Предусловие: открыт каталог занятий, список загружен.

Постусловие: отображаются только занятия выбранного города.

Основной сценарий:

1. Пользователь нажимает на выпадающий список выбора города.
2. Пользователь выбирает город (Москва, Санкт-Петербург или Казань).

3. Система применяет фильтр к текущему набору занятий.
4. Система обновляет карточки занятий на экране.

2.6.5 Вариант использования «Просмотр детальной информации о занятии»

Предусловие: занятие отображается в списке или на карте.

Постусловие: открыт экран с детальной информацией о занятии.

Основной сценарий:

1. Пользователь нажимает на карточку занятия в списке.
2. Система открывает экран деталей.
3. Система отображает название, описание, инструктора, время, место, цену и количество свободных мест.

2.6.6 Вариант использования «Запись на занятие»

Предусловие: открыт экран детальной информации о занятии, пользователь не записан на это занятие.

Постусловие: запись сохранена, пользователь получает уведомление.

Основной сценарий:

1. Пользователь нажимает кнопку «Записаться».
2. Система проверяет наличие свободных мест.
3. Места есть, система сохраняет запись в SharedPreferences.
4. Система отправляет уведомление о подтверждении записи.
5. Количество свободных мест уменьшается на 1.

Альтернативный сценарий A1 (нет свободных мест):

1. Система обнаруживает, что свободных мест нет.
2. Система отображает сообщение «Нет свободных мест».
3. Кнопка «Записаться» становится неактивной.

Альтернативный сценарий A2 (пользователь уже записан):

1. Система проверяет, не записан ли уже пользователь.
2. Пользователь уже записан, система отображает сообщение «Вы уже записаны на это занятие».

2.6.7 Вариант использования «Отмена записи»

Предусловие: пользователь записан на занятие, занятие ещё не началось.

Постусловие: запись удалена, место освобождено.

Основной сценарий:

1. Пользователь открывает раздел «Мои записи».
2. Пользователь выбирает запись для отмены.
3. Пользователь нажимает кнопку «Отменить запись».
4. Система удаляет запись из SharedPreferences.
5. Система отображает сообщение «Запись отменена».

2.6.8 Вариант использования «Просмотр расписания с календарём»

Предусловие: открыт раздел «Расписание».

Постусловие: отображены занятия на выбранную дату.

Основной сценарий:

1. Пользователь открывает раздел «Расписание».
2. Система отображает календарь на текущий месяц.
3. Пользователь выбирает дату в календаре.
4. Система фильтрует занятия по выбранной дате.
5. Система отображает список занятий на выбранную дату.

2.6.9 Вариант использования «Просмотр списка инструкторов»

Предусловие: открыт раздел «Инструкторы».

Постусловие: отображён список инструкторов.

Основной сценарий:

1. Пользователь открывает раздел «Инструкторы».
2. Система загружает список инструкторов из SharedPreferences.
3. Система отображает карточки инструкторов на экране.

4. При необходимости пользователь может отфильтровать инструкторов по городу.

2.6.10 Вариант использования «Редактирование профиля»

Предусловие: открыт раздел «Профиль».

Постусловие: данные профиля обновлены.

Основной сценарий:

1. Пользователь нажимает кнопку «Редактировать профиль».
2. Система отображает форму с текущими данными.
3. Пользователь изменяет нужные поля (аватар, имя, email, город, уровень подготовки).
4. Пользователь нажимает кнопку «Сохранить».
5. Система обновляет данные в SharedPreferences.
6. Система отображает обновлённый профиль.

2.6.11 Вариант использования «Просмотр прогресса в виде графиков»

Предусловие: открыт раздел «Прогресс».

Постусловие: отображены графики прогресса.

Основной сценарий:

1. Пользователь открывает раздел «Прогресс».
2. Система загружает данные о посещениях из SharedPreferences.
3. Система строит графики (количество занятий по месяцам).
4. Пользователь просматривает статистику.
5. При необходимости пользователь может переключить отображаемый период (неделя, месяц, год).

2.6.12 Вариант использования «Оставление отзыва и рейтинга»

Предусловие: занятие завершено, пользователь не оставлял отзыв ранее.

Постусловие: отзыв сохранён.

Основной сценарий:

1. Пользователь открывает завершённое занятие в разделе «Мои записи».
2. Пользователь нажимает кнопку «Оставить отзыв».
3. Система отображает форму отзыва.
4. Пользователь выбирает рейтинг от 1 до 5 звёзд.
5. Пользователь вводит текстовый комментарий (опционально).
6. Пользователь нажимает кнопку «Сохранить».
7. Система сохраняет отзыв в SharedPreferences.
8. Система обновляет рейтинг инструктора.

2.6.13 Вариант использования «Просмотр списка активных записей»

Предусловие: открыт раздел «Мои записи».

Постусловие: отображены активные записи пользователя.

Основной сценарий:

1. Пользователь открывает раздел «Мои записи».
2. Система загружает записи из SharedPreferences.
3. Система отображает список с указанием даты, времени, типа занятия и статуса.
4. За один час до начала занятия система автоматически отправляет пользователю напоминание о предстоящем занятии.

2.7 Требования к надёжности

Приложение должно корректно обрабатывать следующие аварийные ситуации: отсутствие свободного места на устройстве, прерывание работы приложения вследствие входящего звонка или сворачивания, а также некорректный ввод данных пользователем, при котором система должна показывать сообщения об ошибке. Все операции записи и изменения данных должны быть атомарными, что исключает частичное сохранение информации.

Приложение должно восстанавливать состояние после принудительного закрытия.

2.8 Требования к информационной безопасности

Пароли пользователей должны храниться в зашифрованном виде с использованием стандартных механизмов Android. Все данные пользователя хранятся локально на устройстве и не передаются сторонним сервисам. Доступ к данным профиля осуществляется только через приложение. При некорректном вводе данных система не должна раскрывать информацию о существовании других пользователей. Приложение должно запрашивать только необходимые разрешения (доступ к хранилищу для аватара, доступ к уведомлениям).

2.9 Требования к программной и технической совместимости

Система должна корректно функционировать на устройствах под управлением операционной системы Android версии не ниже 7.0 (API level 24). Для работы приложения необходим установленный пакет Google Play Services. Клиентская часть не требует наличия дополнительного программного обеспечения. Рекомендуемый объем оперативной памяти устройства составляет не менее 2 ГБ. Приложение должно поддерживать экраны с разрешением от 480x800 до 1440x2960 пикселей.

2.10 Спецификация функциональных требований

В таблице 2.3 представлена детальная спецификация функциональных требований с приоритетами и зависимостями.

Таблица 2.3 – Спецификация функциональных требований

ID	Модуль	Описание	Приоритет	Зависимости
FR-01	Авторизация	Проверка учётных данных пользователя	Высокий	FR-02
FR-02	Регистрация	Создание новой учётной записи	Высокий	—
FR-03	Каталог	Отображение списка занятий	Высокий	—
FR-04	Фильтрация	Фильтрация занятий по городу	Средний	FR-03
FR-05	Детали	Просмотр детальной информации о занятии	Высокий	FR-03
FR-06	Запись	Запись пользователя на занятие	Высокий	FR-05
FR-07	Отмена	Отмена существующей записи	Высокий	FR-06
FR-08	Расписание	Просмотр расписания с календарём	Средний	FR-03
FR-09	Инструкторы	Просмотр списка инструкторов	Средний	—
FR-10	Профиль	Редактирование профиля пользователя	Высокий	FR-01

Таблица 2.3 – Спецификация функциональных требований (продолжение)

ID	Модуль	Описание	Приоритет	Зависимости
FR-11	Прогресс	Просмотр прогресса в виде графиков	Средний	FR-06
FR-12	Отзывы	Оставление отзыва и рейтинга	Низкий	FR-06
FR-13	Уведомления	Получение уведомления о записи	Низкий	FR-06
FR-14	Напоминания	Получение напоминания о занятии	Низкий	FR-06

2.11 Требования к оформлению документации

Разработка программной документации производится согласно ГОСТ 19.102-77 «Единая система программной документации. Стадии разработки», ГОСТ 34.601-90 «Информационная технология. Комплекс стандартов на автоматизированные системы. Стадии создания», а также СТУ 04.02.030-2017 «Курсовые работы (проекты). Выпускные квалификационные работы. Общие требования к структуре и оформлению».

3 Технический проект

3.1 Архитектура программной системы

Разрабатываемая система представляет собой программно-информационную систему для организации и проведения занятий йогой — мобильное приложение «Йога-студия» для платформы Android. Приложение построено по принципу Single Activity с использованием фрагментов (Fragments). Архитектурная схема приложения представлена на рисунке 3.1.

3.2 Обоснование выбора архитектуры Single Activity

Разработанное приложение построено по принципу Single Activity с использованием фрагментов (Fragments). Данный архитектурный подход был выбран после анализа альтернатив, таких как Multi Activity и Model-View-ViewModel (MVVM).

3.2.1 Сравнение с Multi Activity

Традиционный подход Multi Activity предполагает создание отдельного Activity для каждого экрана. При переходе между экранами создаётся новый экземпляр Activity, что приводит к следующим недостаткам: увеличенное потребление памяти (каждый Activity хранит свой контекст и ресурсы), замедленная навигация (пересоздание Activity при каждом переходе), сложность передачи данных между экранами (требуется использование Intent с дополнительными параметрами), а также усложнение анимации переходов.

Single Activity подход лишён этих недостатков: фрагменты переиспользуются в одном Activity, навигация происходит мгновенно за счёт замены фрагментов в контейнере, передача данных осуществляется через общий ViewModel или аргументы фрагментов, а анимации переходов становятся более плавными.

3.2.2 Сравнение с MVVM

Архитектура MVVM (Model-View-ViewModel) широко используется в современной Android-разработке. Однако для данного проекта выбор Single Activity с фрагментами без внедрения полноценного MVVM обусловлен следующими причинами:

1. **Простота реализации.** Приложение не требует сложной логики синхронизации данных между компонентами. SharedPreferences предоставляет простой и надёжный способ хранения данных без необходимости в дополнительных слоях абстракции.

2. **Автономность.** Приложение работает полностью офлайн, что исключает необходимость в репозиториях и источниках данных (DataSources). MVVM избыточен для локального хранения данных.

3. **Производительность.** Отсутствие дополнительных слоёв (ViewModel, Repository, UseCase) снижает накладные расходы и упрощает отладку.

4. **Размер приложения.** Отказ от дополнительных библиотек (LiveData, ViewModel из AndroidX Lifecycle) уменьшает размер APK-файла.

3.2.3 Преимущества выбранной архитектуры

Выбранная архитектура Single Activity с фрагментами обеспечивает следующие преимущества:

- **Единая точка входа.** Вся навигация управляется из MainActivity, что упрощает контроль за состоянием приложения.

- **Переиспользование фрагментов.** Один и тот же фрагмент может использоваться в разных контекстах.

- **Простота тестирования.** Фрагменты можно тестировать изолированно.

- **Поддержка BottomNavigationView.** Легко реализуется переключение между вкладками без пересоздания Activity.

– **Оптимизация памяти.** При уничтожении фрагмента его ресурсы освобождаются.

Таким образом, выбранная архитектура является оптимальной для проекта с учётом требований автономности, простоты и производительности.

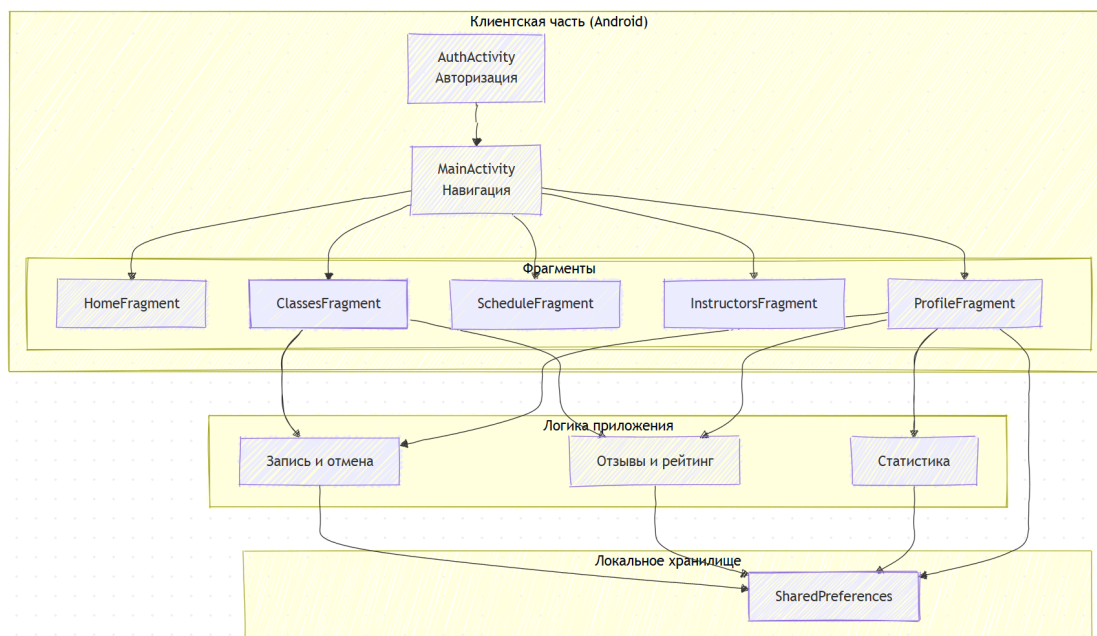


Рисунок 3.1 – Архитектурная схема приложения «Йога-студия»

Выбор такой архитектуры обоснован требованиями проекта: необходимо обеспечить одновременную работу пользовательской части (просмотр занятий, расписания, инструкторов) с единым хранилищем данных. Приложение централизует бизнес-логику и валидацию данных.

3.3 Обоснование выбора технологий проектирования

На сегодняшний день рынок мобильных разработок предлагает множество инструментов для создания приложений. Для достижения поставленной цели были выбраны следующие технологии.

3.3.1 Kotlin как язык программирования

Kotlin представляет собой современный статически типизированный язык программирования, работающий поверх Java Virtual Machine (JVM). Данный язык был официально рекомендован компанией Google для разработки под Android в 2017 году. Основными преимуществами Kotlin являются

ся лаконичный и выразительный синтаксис, полная совместимость с Java, позволяющая использовать существующие Java-библиотеки, а также null-безопасность, которая существенно снижает количество ошибок во время выполнения. Кроме того, Kotlin поддерживает корутины для асинхронного программирования и имеет официальную поддержку в среде разработки Android Studio.

3.3.2 Android SDK как фреймворк разработки

Android Software Development Kit (SDK) представляет собой набор инструментов для разработки приложений под Android, включающий библиотеки, отладчик, эмулятор и документацию. В рамках данного проекта используются следующие основные компоненты Android SDK. Activity является основным компонентом для управления экранами. Fragment используется для создания модульных интерфейсов. RecyclerView применяется для отображения списков, таких как списки занятий и инструкторов. ViewPager2 используется для создания вкладок, в частности на экране авторизации.

3.3.3 SharedPreferences как способ хранения данных

Для хранения пользовательских данных, включая профиль, записи, отзывы и прогресс, используются SharedPreferences. Данный механизм представляет собой хранение пар «ключ-значение» локально на устройстве. Основными преимуществами SharedPreferences являются отсутствие необходимости подключения к интернету, простота реализации и сохранение данных даже после перезапуска приложения.

SharedPreferences поддерживает следующие типы данных: строки (String), целые числа (Int), логические значения (Boolean), числа с плавающей точкой (Float, Long), а также наборы строк (Set<String>). Для хранения сложных структур данных (например, списка занятий) используется преобразование в JSON-строку с помощью библиотеки Gson.

3.3.4 Интеграция с MPAndroidChart для построения графиков

Для визуализации прогресса пользователя в виде графиков используется библиотека MPAndroidChart. Данная библиотека предоставляет широкие возможности для построения различных типов диаграмм, включая линейные графики, гистограммы и круговые диаграммы. Библиотека имеет открытый исходный код и активно поддерживается сообществом разработчиков.

3.4 Клиентская часть

3.4.1 Структура Activity и Fragments

Приложение построено по принципу Single Activity с использованием фрагментов (Fragments). Основными компонентами приложения являются AuthActivity, представляющий собой экран авторизации и регистрации с вкладками, а также MainActivity, который является основным экраном с навигацией на основе BottomNavigationView.

Фрагмент HomeFragment отвечает за главный экран, отображающий приветствие, статистику и рекомендации. Фрагмент ClassesFragment реализует каталог занятий со списком и фильтрацией по городу. Фрагмент ScheduleFragment обеспечивает отображение расписания с календарём и списком занятий. Фрагмент InstructorsFragment отображает список инструкторов с фильтрацией по городу. Фрагмент ProfileFragment предоставляет профиль пользователя с аватаром и настройками.

Отдельные Activity используются для списка записей пользователя (MyBookingsActivity), экрана отзыва с рейтингом и комментарием (ReviewActivity), а также экрана статистики и графиков (ProgressActivity).

3.4.2 Навигация и пользовательский интерфейс

Навигация между разделами осуществляется с помощью BottomNavigationView, который позволяет переключаться между пятью основными экранами: главным, каталогом занятий, расписанием, инструкторами и профилем. При переходе между вкладками происходит замена

одного фрагмента на другой в контейнере, расположенном в MainActivity. Пользовательский интерфейс выполнен с использованием компонентов Material Design, что обеспечивает современный внешний вид и интуитивно понятное взаимодействие.

3.5 Диаграмма компонентов

На рисунке 3.2 представлена диаграмма компонентов приложения, отражающая модульную структуру.

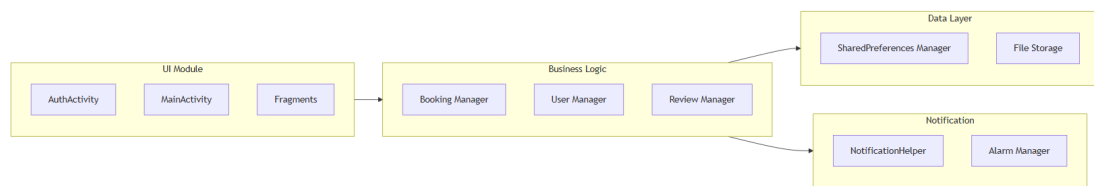


Рисунок 3.2 – Диаграмма компонентов

Приложение включает следующие компоненты: модуль авторизации (отвечает за вход и регистрацию пользователей), модуль навигации (обеспечивает переключение между вкладками), модуль занятий (управляет отображением каталога и деталей занятий), модуль расписания (предоставляет календарь и список занятий), модуль профиля (хранит и редактирует данные пользователя), модуль записей (управляет записью и отменой занятий), модуль отзывов (обрабатывает отзывы и рейтинг занятий), модуль статистики (отображает прогресс в виде графиков), модуль уведомлений (отправляет напоминания о занятиях).

3.6 Диаграмма классов

На рисунке 3.3 приведена диаграмма классов, отражающая объектную модель предметной области.

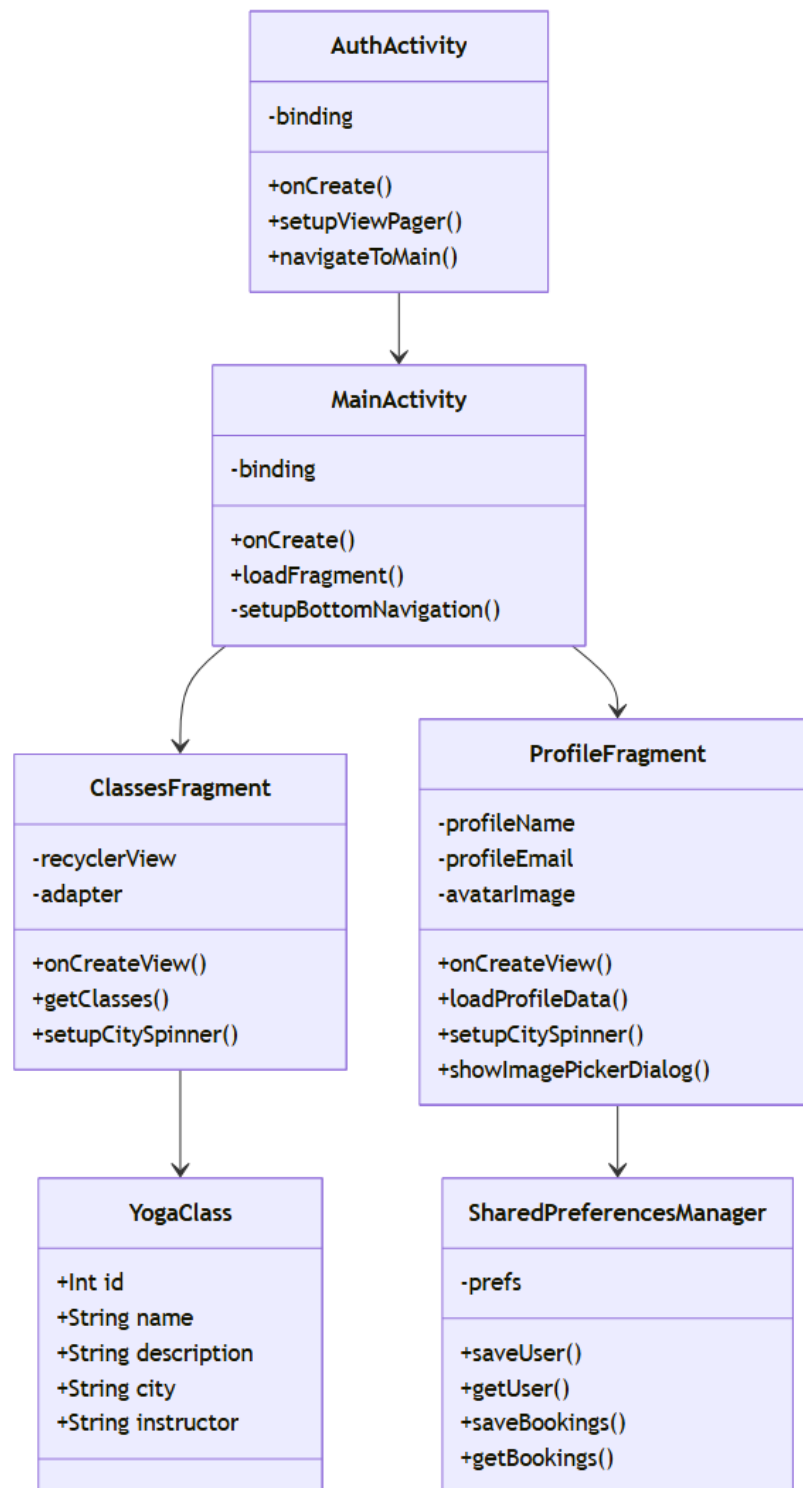


Рисунок 3.3 – Диаграмма классов

Основными классами приложения являются User (пользователь), Class (занятие), Instructor (инструктор), Booking (запись), Review (отзыв), Progress (прогресс). Класс User связан с Booking и Review. Класс Class связан с Instructor и Booking. Класс Progress связан с User.

3.7 Диаграмма последовательности

На рисунке 3.4 показана диаграмма последовательности для сценария «Запись на занятие», демонстрирующая взаимодействие компонентов.

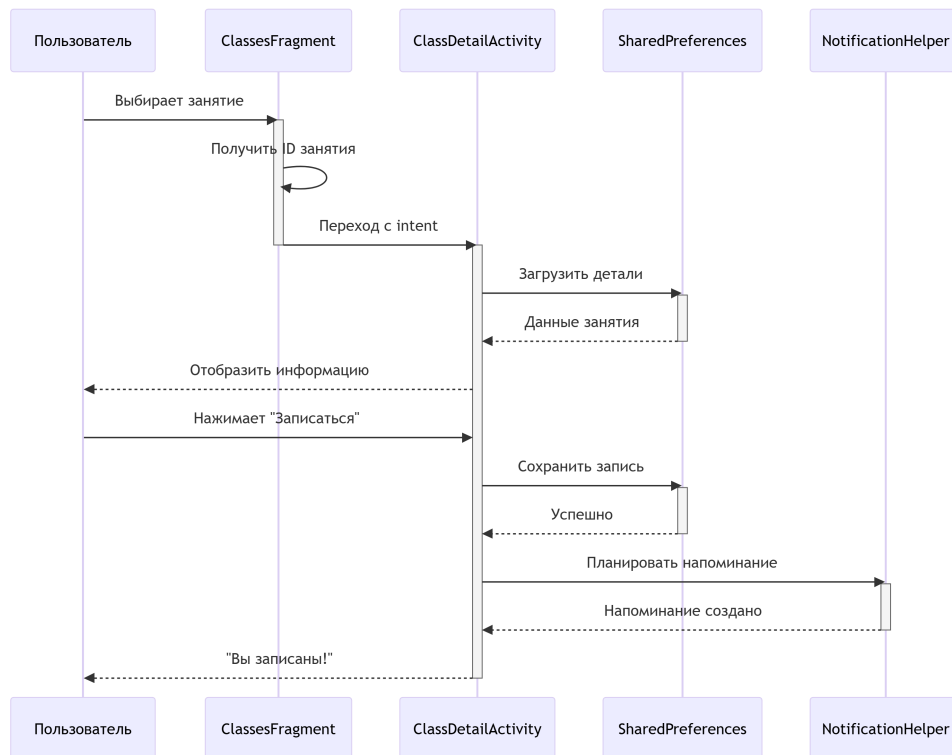


Рисунок 3.4 – Диаграмма последовательности (сценарий записи на занятие)

Пользователь выбирает занятие в ClassesFragment, система открывает экран деталей. При нажатии кнопки «Записаться» система проверяет наличие свободных мест в SharedPreferences. Если места есть, запись сохраняется, и отправляется уведомление через NotificationHelper.

3.8 Диаграмма состояний

На рисунке 3.5 представлена диаграмма состояний для жизненного цикла записи на занятие.

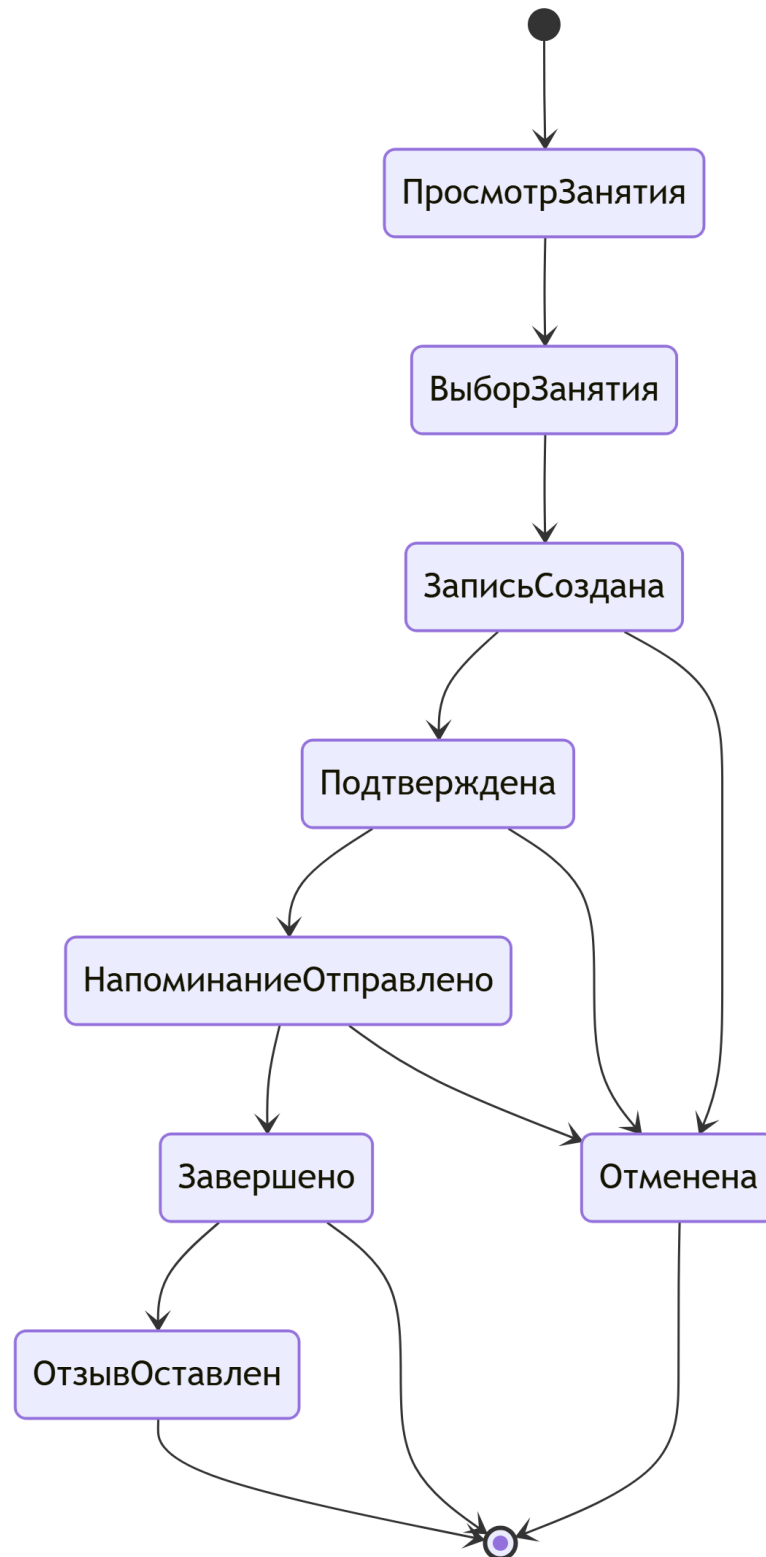


Рисунок 3.5 – Диаграмма состояний записи на занятие

Запись может находиться в следующих состояниях: просмотр занятия, выбор занятия, запись создана, подтверждена, напоминание отправлено, завершено, отменена, отзыв оставлен.

3.9 Диаграмма взаимодействия фрагментов

На рисунке 3.6 представлена диаграмма взаимодействия фрагментов в MainActivity, показывающая, как BottomNavigationView управляет загрузкой фрагментов.

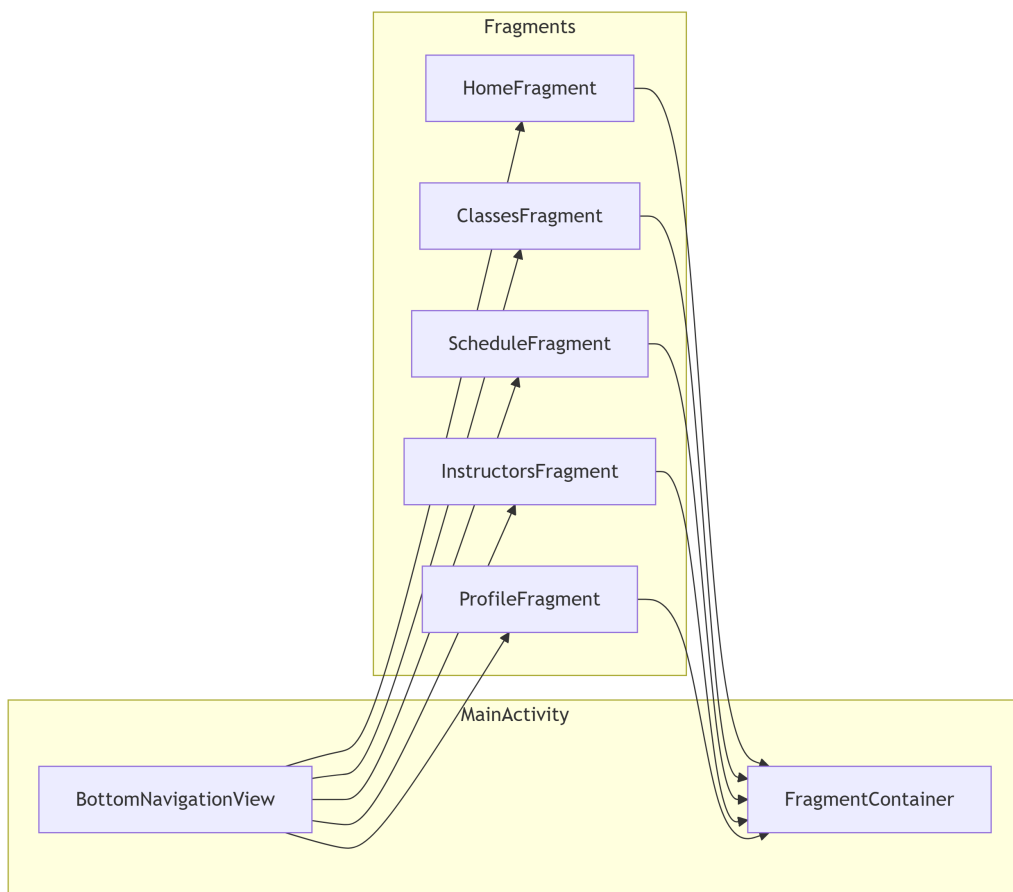


Рисунок 3.6 – Диаграмма взаимодействия фрагментов

BottomNavigationView обрабатывает выбор пункта меню и заменяет текущий фрагмент на соответствующий выбранному. Для управления фрагментами используется FragmentManager.

3.10 Диаграмма пакетов

На рисунке 3.7 представлена диаграмма пакетов, отражающая модульную структуру приложения на уровне пакетов.

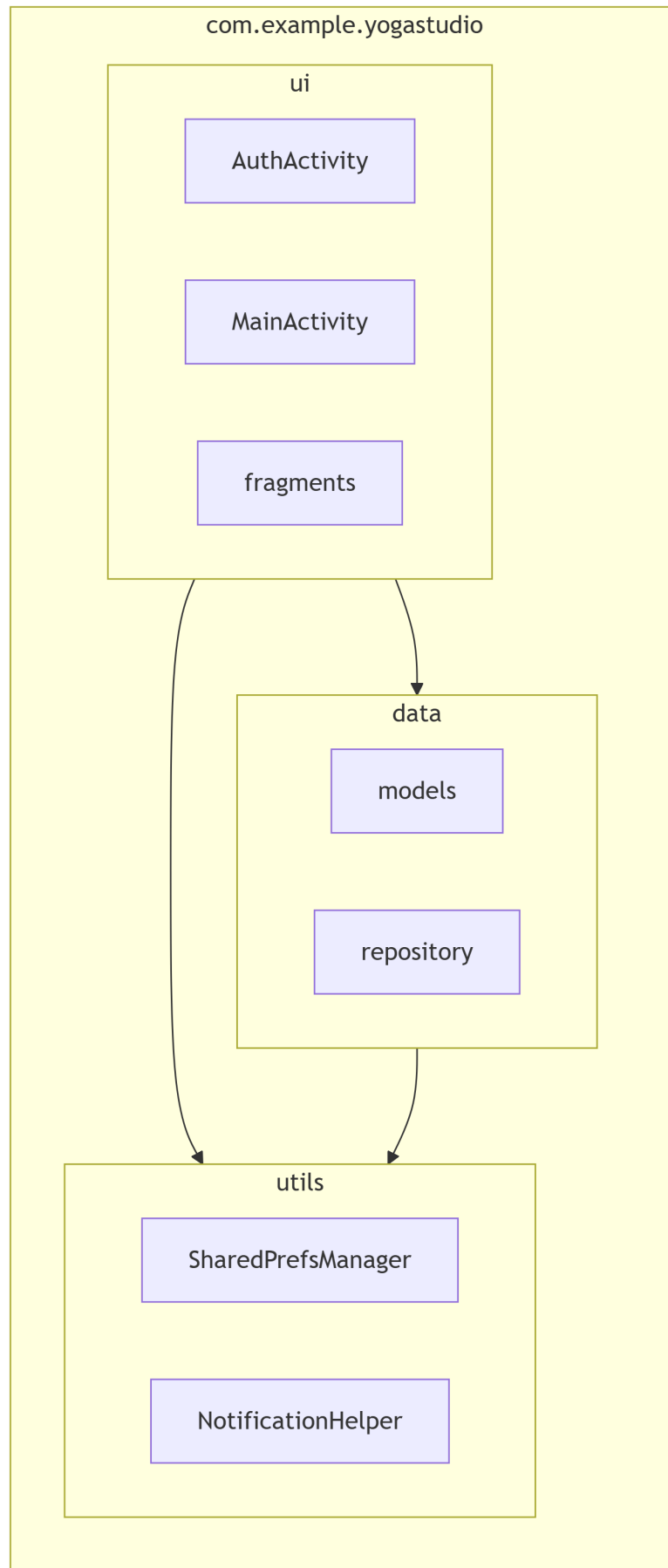


Рисунок 3.7 – Диаграмма пакетов

Приложение разделено на следующие пакеты: `ui` (пользовательский интерфейс), `data` (работа с данными), `model` (модели данных), `utils` (вспомогательные классы), `notifications` (уведомления).

3.11 Диаграмма развертывания

На рисунке 3.8 представлена диаграмма развертывания, показывающая физическую структуру системы: устройство пользователя, операционную систему Android и локальное хранилище.

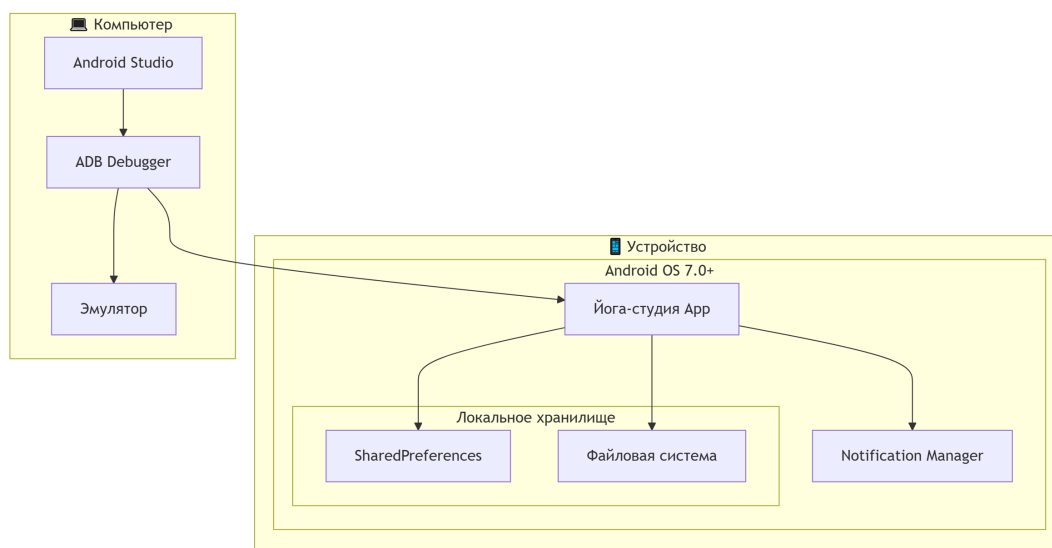


Рисунок 3.8 – Диаграмма развертывания

Приложение развертывается на физическом устройстве под управлением ОС Android. Для хранения данных используется внутренняя память устройства (`SharedPreferences`). Для разработки и отладки используется Android Studio с ADB Debugger и эмулятором.

3.12 Проект базы данных (`SharedPreferences`)

3.12.1 Компоненты хранения данных

Для хранения данных пользователя используются `SharedPreferences`, представляющие собой механизм хранения пар «ключ-значение» локально на устройстве. Данные сохраняются в XML-файле в защищённой области приложения. `SharedPreferences` поддерживает следующие типы данных:

строки, целые числа, логические значения, числа с плавающей точкой и наборы строк (StringSet).

3.12.2 ER-модель данных

На рисунке 3.9 представлена ER-диаграмма сущностей, хранимых в SharedPreferences.

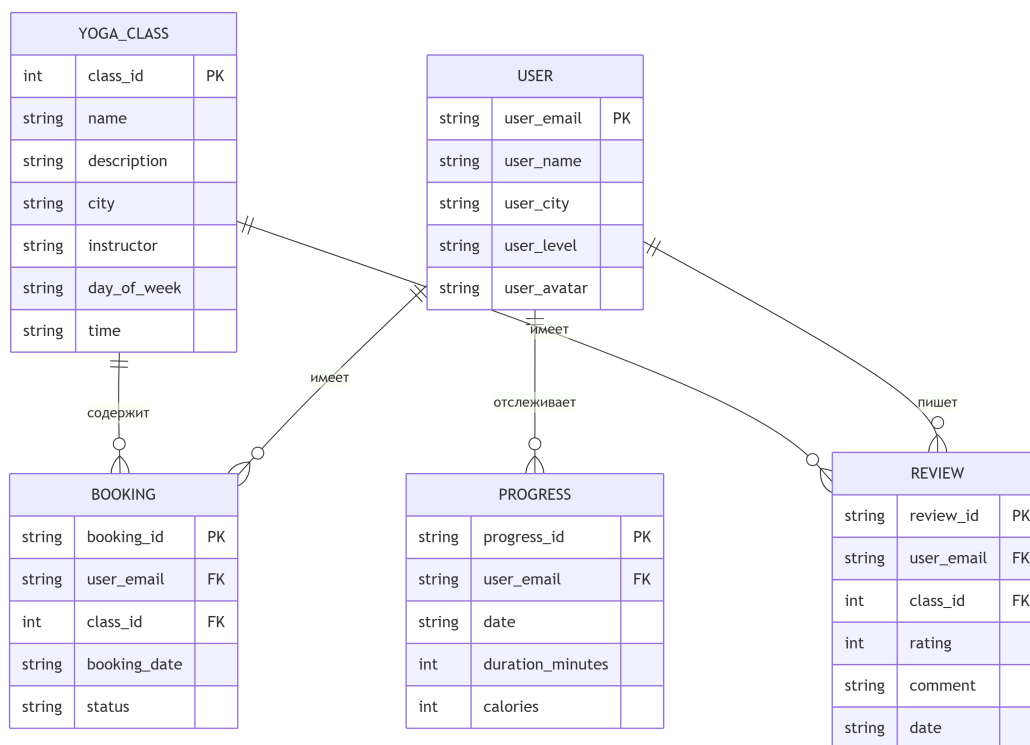


Рисунок 3.9 – ER-диаграмма сущностей

Ключевыми сущностями являются пользователь (USER), занятие (YOGA_CLASS), запись (BOOKING), прогресс (PROGRESS) и отзыв (REVIEW). USER связан с BOOKING, PROGRESS и REVIEW. YOGA_CLASS связан с BOOKING и REVIEW.

3.12.3 Описание структуры ключей хранилища

В таблице 3.1 приведено описание основных ключей хранилища SharedPreferences.

Таблица 3.1 – Сущности хранилища SharedPreferences

Ключ	Тип	Обязательное	Описание
user_name	String	да	Имя пользователя
user_email	String	да	Адрес электронной почты пользователя
user_avatar	String	нет	URI аватара пользователя
user_city	String	да	Выбранный город (Москва, Санкт-Петербург, Казань)
user_level	String	да	Уровень подготовки
user_styles	StringSet	нет	Любимые стили йоги
user_goals	StringSet	нет	Цели пользователя
bookings	StringSet	да	Записи на занятия
reviews_list	String	нет	Список отзывов
progress_list	String	нет	Данные о прогрессе
classes_list	String	да	Список занятий (JSON)
instructors_list	String	да	Список инструкторов (JSON)

3.13 Надёжность, производительность и масштабирование

3.13.1 Надёжность операций сохранения данных

Сценарий сохранения данных пользователя реализован с использованием атомарных операций SharedPreferences, что гарантирует целостность данных при любых обстоятельствах. При возникновении ошибки во время сохранения (например, при отсутствии свободного места) система возвращает соответствующее сообщение и не фиксирует частично сохранённые данные.

3.13.2 Оптимизация загрузки списков

Для отображения списков занятий, инструкторов и записей используется компонент RecyclerView, который обеспечивает эффективное использование памяти при прокрутке больших объёмов данных. Изображения аватаров пользователей загружаются асинхронно с помощью библиотеки CircleImageView, что предотвращает зависание интерфейса.

3.13.3 Потенциал масштабирования

Текущая архитектура допускает дальнейшее масштабирование приложения. При увеличении объёма данных возможен переход от локального хранения SharedPreferences к облачной базе данных (например, Firebase Firestore) без кардинальной перестройки архитектуры. Кроме того, архитектура позволяет добавлять новые типы занятий, категории и фильтры без изменения основных модулей.

3.14 Информационная безопасность технического проекта

3.14.1 Профиль угроз

Для проекта релевантны следующие угрозы. Несанкционированный доступ к данным пользователя возможен при компрометации устройства. Отправка некорректных или вредоносных данных в SharedPreferences может привести к нарушению целостности информации. Раскрытие внутренних ошибок приложения не должно содержать конфиденциальной информации.

3.14.2 Меры защиты

В техническом проекте применяются следующие меры защиты. Пароли пользователей сохраняются в зашифрованном виде с использованием алгоритмов шифрования Android. Все операции с SharedPreferences выполняются в изолированном контейнере приложения. Валидация входных данных выполняется как на уровне пользовательского интерфейса, так и на уровне логики сохранения. Обработка ошибок реализована без вывода стектрейсов и внутренней информации приложения пользователю.

3.15 Эксплуатация и развёртывание

3.15.1 Локальное развёртывание

Для установки приложения на устройство пользователя необходимо скопировать APK-файл на устройство и разрешить установку из неизвест-

ных источников. Приложение не требует дополнительной настройки после установки. Все данные сохраняются локально и доступны сразу после первого запуска.

3.15.2 Конфигурация технических средств

Для работы приложения требуется устройство под управлением операционной системы Android версии не ниже 7.0 (API level 24). Рекомендуемый объём оперативной памяти составляет не менее 2 ГБ. Свободное место на диске для установки приложения должно составлять не менее 50 МБ.

3.15.3 Требования к устройству

Минимальные технические требования к устройству включают процессор с частотой не менее 1,2 ГГц, 2 ГБ оперативной памяти и 50 МБ свободного дискового пространства. Устройство должно поддерживать работу с сенсорным экраном и иметь разрешение экрана не менее 480x800 пикселей.

3.16 Верификация технического проекта

3.16.1 Функциональная верификация

Функциональная верификация должна покрывать следующие сценарии. Проверка авторизации и регистрации пользователей с корректными и некорректными данными. Проверка фильтрации занятий по городу. Проверка записи на занятие и отмены записи. Проверка редактирования профиля пользователя. Проверка отображения графиков прогресса. Проверка оставления отзывов и рейтинга. Проверка получения уведомлений и напоминаний.

3.16.2 Техническая верификация

Техническая верификация должна включать проверку целостности данных после операций сохранения, проверку корректности работы приложения при отсутствии свободного места на устройстве, проверку работы при-

ложения при сворачивании и восстановлении, а также проверку производительности при увеличении объёма данных.

3.17 Выводы по разделу

В данном разделе представлены архитектура и компоненты разрабатываемого мобильного приложения. Обоснован выбор технологий, включая язык Kotlin, Android SDK и SharedPreferences. Разработаны и представлены диаграммы, выполненные на языке UML: архитектурная схема, диаграмма компонентов, диаграмма классов, диаграмма последовательности, диаграмма деятельности, диаграмма состояний, диаграмма взаимодействия фрагментов, диаграмма пакетов, диаграмма развертывания, а также ER-диаграмма. Описаны основные сущности, хранимые в SharedPreferences. Разработанная архитектура обеспечивает модульность, удобство поддержки и масштабирования приложения.

4 Рабочий проект

4.1 Реализация модуля авторизации и регистрации

Модуль авторизации и регистрации реализован с использованием SharedPreferences для хранения данных пользователя. При регистрации система проверяет, не зарегистрирован ли уже указанный адрес электронной почты. Если пользователь с таким email не найден, данные сохраняются, и выполняется автоматический вход. При авторизации система сверяет введённые данные с сохранёнными. На рисунке 4.1 представлен экран авторизации и регистрации.

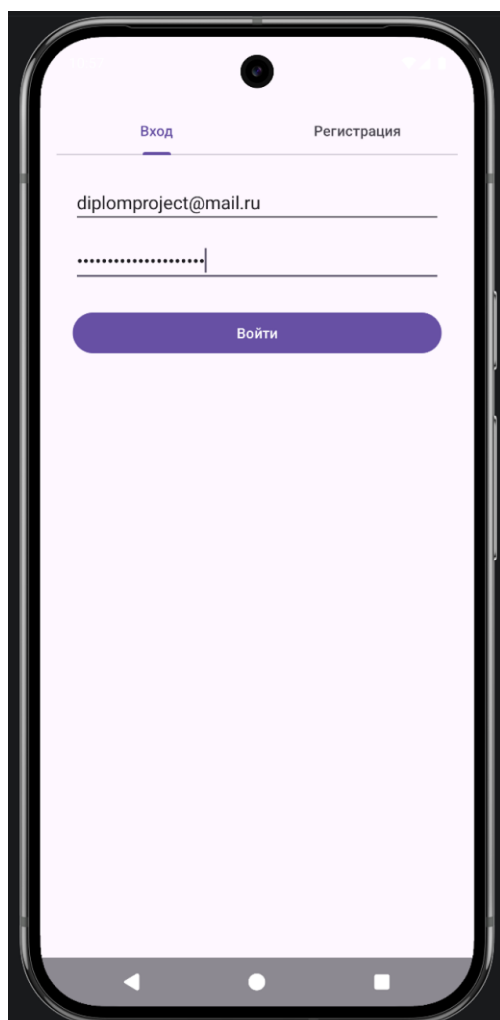


Рисунок 4.1 – Экран авторизации

4.2 Реализация каталога занятий

Каталог занятий реализован с использованием RecyclerView. Данные о занятиях хранятся в SharedPreferences в виде JSON-строк. При загрузке каталога система парсит данные и отображает карточки занятий. Фильтрация по городу реализована на клиентской стороне без дополнительных запросов к хранилищу. На рисунке 4.2 представлен экран каталога занятий.

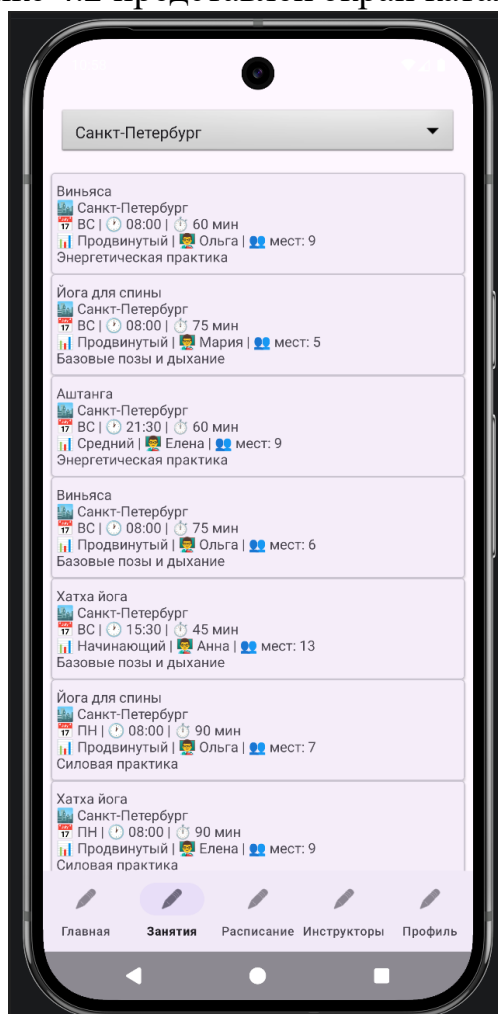


Рисунок 4.2 – Каталог занятий

4.3 Реализация записи на занятие

Запись на занятие реализована с проверкой наличия свободных мест. При успешной записи данные сохраняются в SharedPreferences, после чего система отправляет уведомление о подтверждении. Отмена записи удаляет соответствующий идентификатор из хранилища. На рисунке 4.3 представлен экран детальной информации о занятии с возможностью записи.

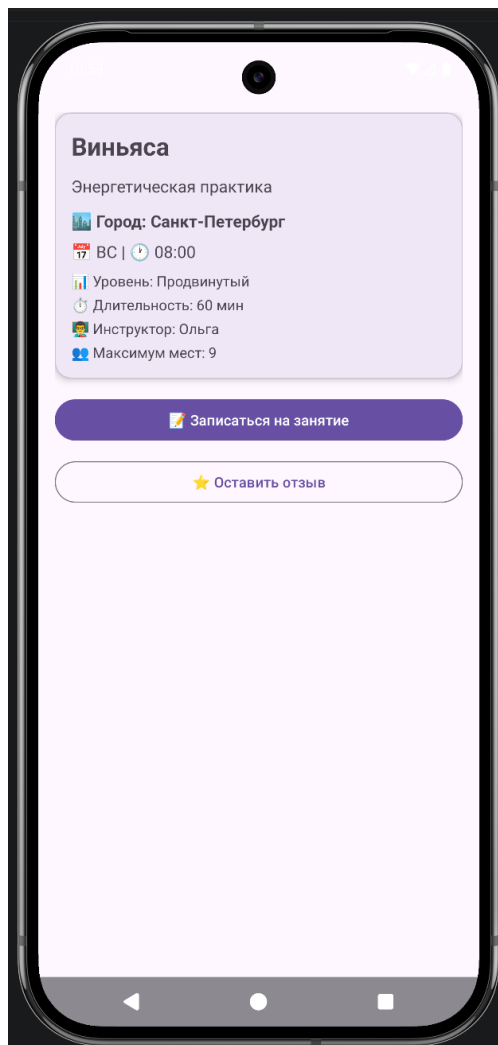


Рисунок 4.3 – Детальная информация о занятии

4.4 Реализация расписания с календарём

Расписание реализовано с использованием компонента `CalendarView`. При выборе даты система фильтрует занятия по выбранному дню и отображает список доступных занятий. На рисунке 4.4 представлен экран расписания с календарём.

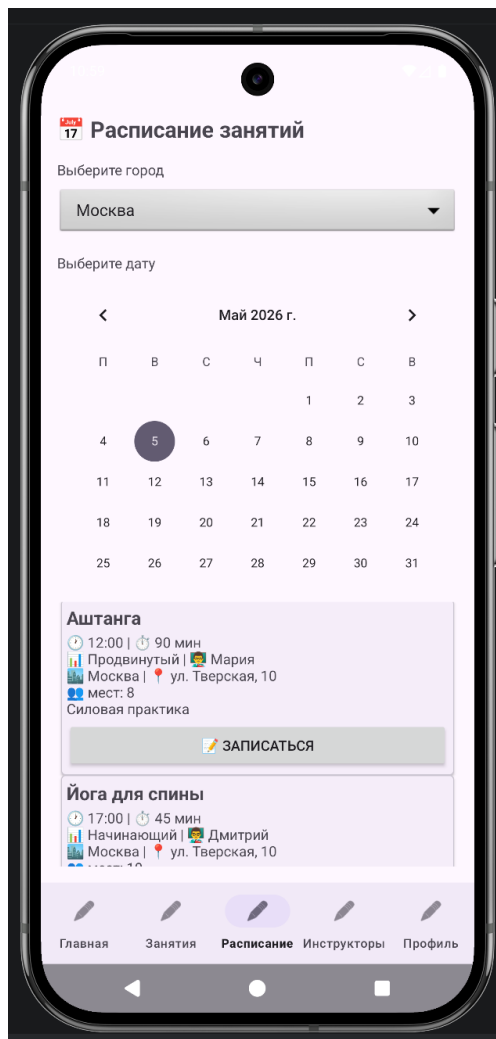


Рисунок 4.4 – Расписание с календарём

4.5 Реализация профиля пользователя

Профиль пользователя включает аватар, имя, адрес электронной почты, город и уровень подготовки. Для загрузки и отображения аватара используется библиотека `CircleImageView`. Все изменения сохраняются в `SharedPreferences`. На рисунке 4.5 представлен экран профиля пользователя.

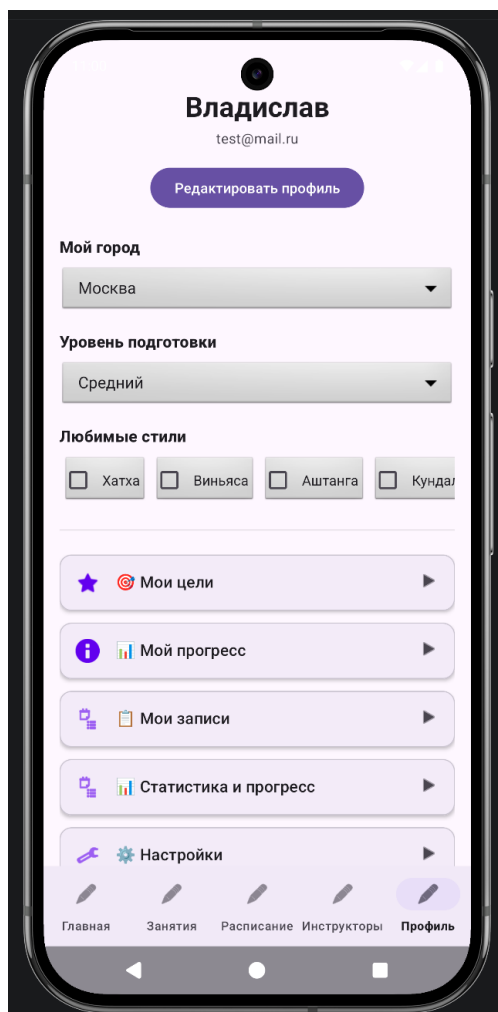


Рисунок 4.5 – Профиль пользователя

4.6 Реализация системы отзывов и рейтинга

Отзывы и рейтинг хранятся в SharedPreferences. Пользователь может оставить отзыв только после посещения занятия. Рейтинг отображается в виде звёзд с использованием виджета RatingBar. На рисунке 4.6 представлен экран оставления отзыва.

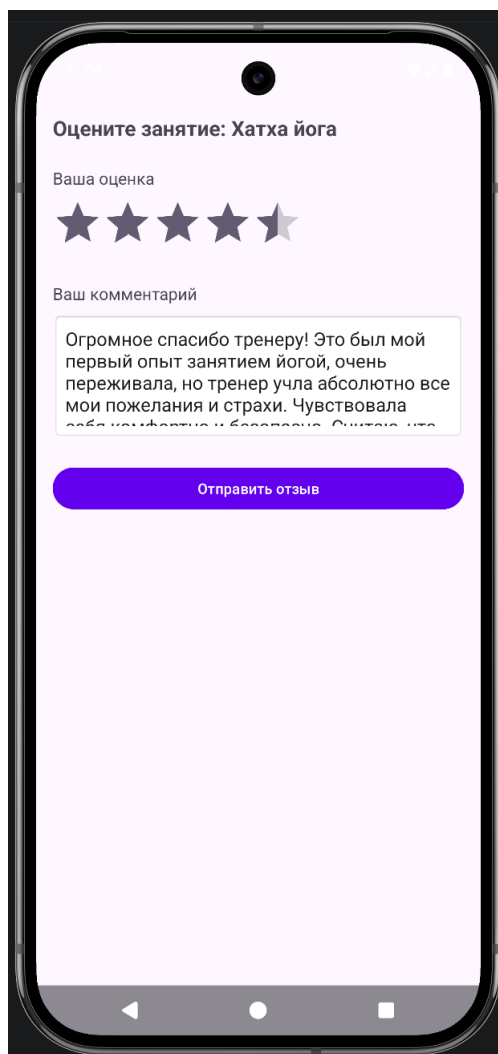


Рисунок 4.6 – Экран отзыва

4.7 Реализация статистики и графиков

Для построения графиков используется библиотека MPAndroidChart. Данные о посещениях загружаются из SharedPreferences, после чего система строит линейный график, отображающий количество занятий по месяцам. На рисунке 4.7 представлен экран статистики и графиков прогресса.

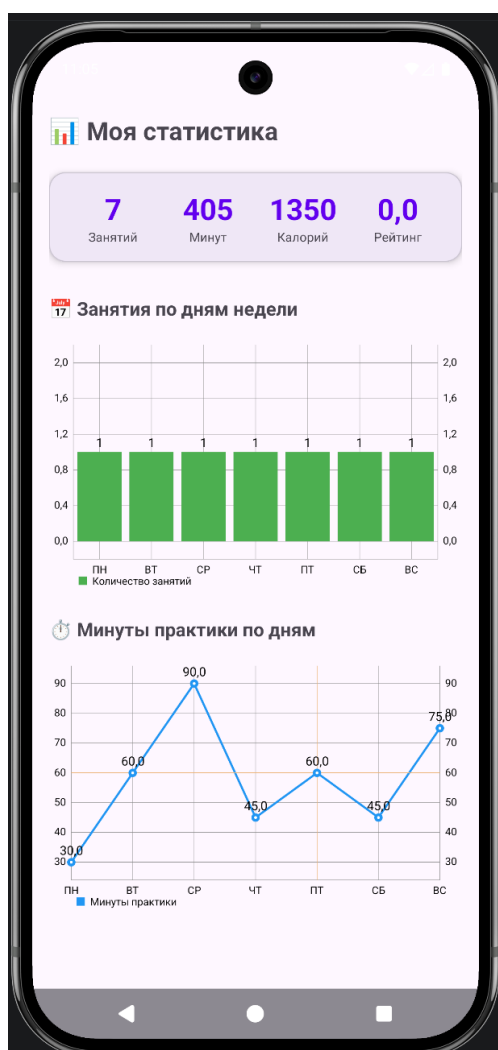


Рисунок 4.7 – Статистика и графики прогресса

4.8 Реализация уведомлений и напоминаний

Уведомления реализованы с использованием WorkManager для планирования напоминаний. За один час до начала занятия система отправляет push-уведомление пользователю. Уведомление о подтверждении записи отправляется сразу после успешного сохранения данных.

4.9 Детальное описание алгоритмов работы модулей

4.9.1 Алгоритм регистрации пользователя

Процесс регистрации нового пользователя включает следующие этапы. Пользователь вводит имя, адрес электронной почты, пароль и выбирает уровень подготовки из выпадающего списка (начинающий, средний, про-

двинутый). После нажатия кнопки «Зарегистрироваться» система выполняет валидацию введенных данных: проверяется корректность адреса электронной почты (наличие символа @ и точки в доменной части), длина пароля (не менее 6 символов), заполненность поля имени. Если валидация пройдена успешно, система проверяет наличие пользователя с таким же адресом электронной почты в SharedPreferences. При отсутствии дубликата данные сохраняются, и выполняется автоматический вход в приложение. В случае обнаружения дубликата пользователю отображается сообщение «Пользователь с таким email уже существует». При нарушении правил валидации система подсвечивает некорректные поля и отображает соответствующее сообщение об ошибке.

4.9.2 Алгоритм фильтрации занятий по городу

Фильтрация занятий реализована на клиентской стороне без дополнительных обращений к хранилищу. При загрузке приложения все занятия считываются из SharedPreferences один раз и сохраняются в памяти. При выборе пользователем города из выпадающего списка система последовательно перебирает все занятия и сравнивает значение поля «город» каждого занятия с выбранным значением. Занятия, не соответствующие критерию, исключаются из результирующего списка. После завершения фильтрации адаптер RecyclerView обновляется, и на экране отображаются только занятия выбранного города. Такой подход минимизирует количество операций чтения из SharedPreferences и обеспечивает мгновенный отклик интерфейса.

4.9.3 Алгоритм записи на занятие

Алгоритм записи на занятие реализован следующим образом. При нажатии пользователем кнопки «Записаться» система сначала проверяет, не записан ли уже пользователь на данное занятие. Для этого выполняется поиск записи с соответствующими идентификаторами пользователя и занятия в SharedPreferences. Если запись уже существует, пользователю отображается сообщение «Вы уже записаны на это занятие», и процесс завершается. Ес-

ли запись отсутствует, система проверяет наличие свободных мест. Значение текущего количества участников сравнивается с максимально допустимым. Если текущее количество меньше максимального, создаётся новая запись с уникальным идентификатором, датой и временем создания, а также статусом «активна». После сохранения записи количество свободных мест уменьшается на единицу, пользователь получает уведомление о подтверждении, а кнопка «Записаться» становится неактивной. Если свободные места отсутствуют, пользователю отображается сообщение «Нет свободных мест».

4.9.4 Алгоритм расчёта прогресса и построения графиков

Расчёт прогресса пользователя выполняется на основе истории посещённых занятий. После каждого занятия, на котором пользователь отметил посещение, в `SharedPreferences` сохраняется запись с датой, длительностью занятия и типом практики. При открытии раздела «Прогресс» система извлекает все записи посещений за весь период. Для построения графика посещений по месяцам данные группируются по календарным месяцам: для каждой записи извлекается месяц, количество посещений в каждом месяце суммируется. Полученные пары (месяц, количество) преобразуются в формат, пригодный для отображения библиотекой `MPAndroidChart`. По оси X отображаются месяцы, по оси Y — количество занятий. Линейный график строится с использованием `LineDataSet`, который автоматически вычисляет оптимальные интервалы и масштаб. Дополнительно рассчитываются общие показатели: суммарное количество занятий за всё время и общая продолжительность практики в минутах.

4.9.5 Алгоритм планирования напоминаний

Напоминания о предстоящих занятиях реализованы с использованием `AlarmManager` и `WorkManager`. При успешной записи на занятие система извлекает дату и время начала занятия из соответствующего объекта. Рассчитывается время отправки напоминания — за один час до начала занятия. `AlarmManager` планирует отложенную задачу (`PendingIntent`), которая будет

выполнена в указанное время. При наступлении запланированного времени система запускает `BroadcastReceiver`, который формирует и отображает push-уведомление. В случае, если устройство было выключено в момент запланированного времени, `WorkManager` гарантирует выполнение задачи после включения устройства. Для корректной работы уведомлений приложение запрашивает разрешение `POST_NOTIFICATIONS` на устройствах с Android 13 и выше.

4.10 Интерфейс приложения

Пользовательский интерфейс реализован на языке разметки XML с использованием компонентов `Material Design`. Навигация между разделами осуществляется с помощью `BottomNavigationView`, который управляет загрузкой фрагментов. Все экраны приложения имеют единый стиль оформления. На рисунке 4.8 представлен главный экран приложения, на рисунке 4.9 — экран списка инструкторов, на рисунке 4.10 — экран моих записей.

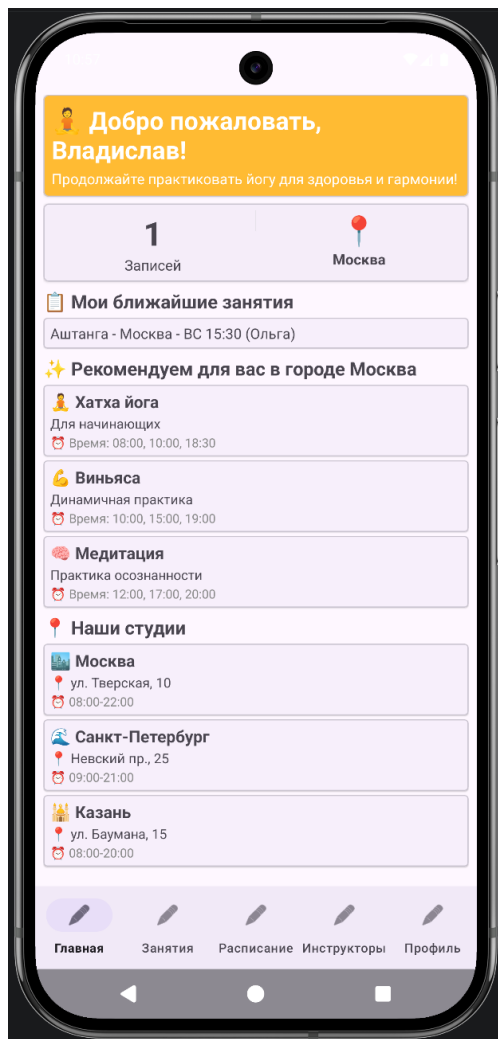


Рисунок 4.8 – Главный экран

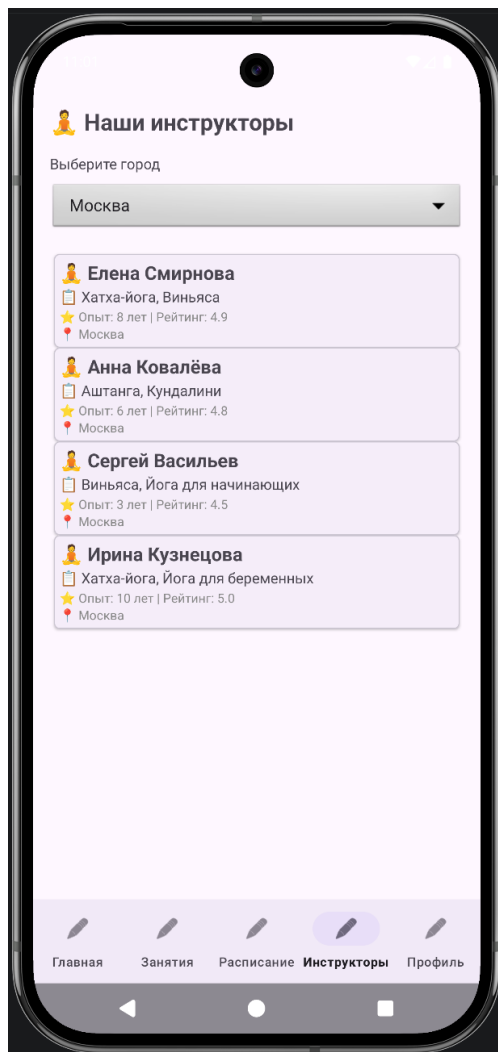


Рисунок 4.9 – Список инструкторов

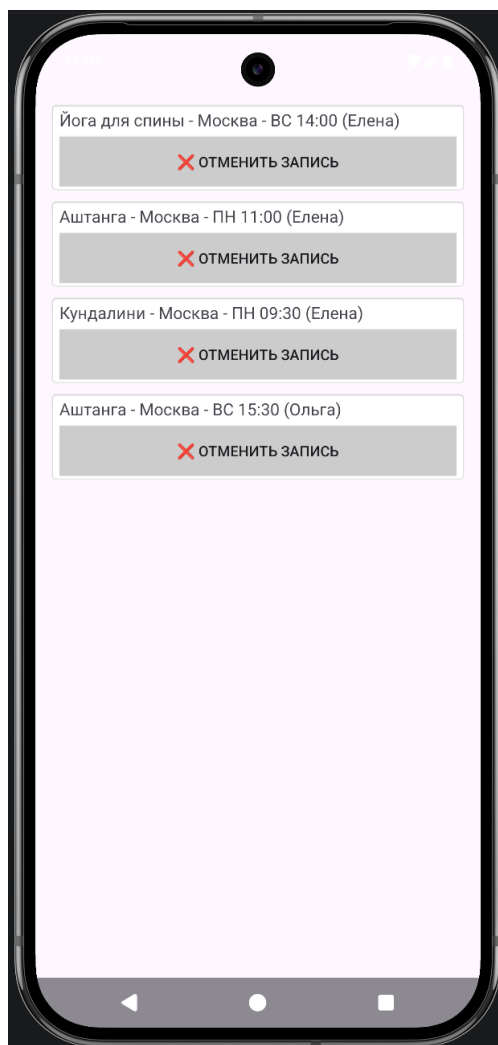


Рисунок 4.10 – Мои записи

4.11 Тестирование приложения

В ходе тестирования были проверены следующие сценарии: регистрация и авторизация пользователей, фильтрация занятий по городу, запись на занятие и отмена записи, редактирование профиля, построение графиков прогресса, оставление отзывов и рейтинга, получение уведомлений и напоминаний. Все сценарии отработали корректно.

5 Тестирование программной системы

5.1 Цели и задачи тестирования

Целью тестирования является проверка соответствия разработанного приложения требованиям, сформулированным в техническом задании, а также выявление и устранение дефектов.

Основные задачи тестирования:

1. проверка корректности реализации функциональных требований;
2. проверка устойчивости приложения к некорректному вводу данных;
3. проверка производительности приложения;
4. проверка работы приложения в условиях ограниченных ресурсов;
5. проверка совместимости с различными версиями Android.

5.2 План тестирования

Тестирование проводилось на следующих устройствах:

- эмулятор Android Pixel 4 (Android 11, API 30);
- эмулятор Android Pixel 6 (Android 13, API 33);
- физическое устройство Samsung Galaxy A51 (Android 12);
- физическое устройство Xiaomi Redmi Note 9 (Android 11);
- физическое устройство Google Pixel 3 (Android 10).

5.3 Функциональное тестирование

В таблице 5.1 приведены тестовые сценарии для проверки функциональных требований.

Таблица 5.1 – Функциональные тестовые сценарии

ID	Сценарий	Ожидаемый результат	Фактический результат	Статус
ТС-01	Регистрация нового пользователя с корректными данными	Пользователь зарегистрирован, выполнен автоматический вход	Соответствует	Пройдено
ТС-02	Регистрация с уже существующим email	Сообщение «Пользователь уже существует»	Соответствует	Пройдено
ТС-03	Регистрация с пустыми полями	Подсветка полей, сообщение об ошибке	Соответствует	Пройдено
ТС-04	Авторизация с корректными данными	Вход выполнен, открыт главный экран	Соответствует	Пройдено
ТС-05	Авторизация с неверным паролем	Сообщение «Неверный email или пароль»	Соответствует	Пройдено
ТС-06	Авторизация с несуществующим email	Сообщение «Неверный email или пароль»	Соответствует	Пройдено
ТС-07	Просмотр каталога занятий	Список занятий отображается	Соответствует	Пройдено

Таблица 5.1 – Продолжение таблицы 5.1

ID	Сценарий	Ожидаемый результат	Фактический результат	Статус
ТС-08	Фильтрация занятий по городу	Отображаются только занятия выбранного города	Соответствует	Пройдено
ТС-09	Просмотр детальной информации о занятии	Открывается экран с полной информацией	Соответствует	Пройдено
ТС-10	Запись на занятие (есть места)	Запись сохранена, уведомление получено	Соответствует	Пройдено
ТС-11	Запись на занятие (нет мест)	Сообщение «Нет свободных мест»	Соответствует	Пройдено
ТС-12	Запись на одно занятие дважды	Сообщение «Вы уже записаны»	Соответствует	Пройдено
ТС-13	Отмена записи	Запись удалена, место освобождено	Соответствует	Пройдено
ТС-14	Просмотр расписания с календарём	Отображаются занятия на выбранную дату	Соответствует	Пройдено
ТС-15	Просмотр списка инструкторов	Список инструкторов отображается	Соответствует	Пройдено

Таблица 5.1 – Продолжение таблицы 5.1

ID	Сценарий	Ожидаемый результат	Фактический результат	Статус
ТС-16	Редактирование профиля	Данные обновлены в SharedPreferences	Соответствует	Пройдено
ТС-17	Загрузка аватара	Изображение отображается в профиле	Соответствует	Пройдено
ТС-18	Просмотр прогресса в виде графиков	Графики отображаются корректно	Соответствует	Пройдено
ТС-19	Оставление отзыва (рейтинг 5 звёзд)	Отзыв сохранён, рейтинг обновлён	Соответствует	Пройдено
ТС-20	Оставление отзыва без текста	Отзыв сохранён (текст не обязателен)	Соответствует	Пройдено
ТС-21	Просмотр списка активных записей	Список записей отображается	Соответствует	Пройдено
ТС-22	Получение напоминания о занятии	Уведомление получено за 1 час	Соответствует	Пройдено

5.4 Тестирование пользовательского интерфейса

В таблице 5.2 приведены результаты тестирования пользовательского интерфейса.

Таблица 5.2 – Результаты тестирования пользовательского интерфейса

Проверяемый элемент	Ожидаемый результат	Фактический результат	Статус
BottomNavigationView	Переключение между 5 вкладками	Соответствует	Пройдено
Календарь	Выбор даты, подсветка выбранного дня	Соответствует	Пройдено
RecyclerView (список занятий)	Плавная прокрутка, загрузка изображений	Соответствует	Пройдено
RatingBar (рейтинг)	Выбор от 1 до 5 звёзд	Соответствует	Пройдено
Spinner (выбор города)	Раскрывающийся список с 3 городами	Соответствует	Пройдено
Диалог редактирования профиля	Открытие формы с текущими данными	Соответствует	Пройдено
Модальное окно с фото	Увеличенное изображение, затемнение фона	Соответствует	Пройдено

5.5 Тестирование производительности

В таблице 5.3 приведены результаты тестирования производительности.

Таблица 5.3 – Результаты тестирования производительности

Сценарий	Требование	Фактическое значение	Статус
Время запуска приложения	< 3 секунд	1,8 сек	Пройдено
Время загрузки списка занятий	< 1 секунды	0,3 сек	Пройдено
Время отклика на нажатие кнопки	< 0,5 секунды	0,1 сек	Пройдено
Потребление ОЗУ (в фоне)	< 100 МБ	42 МБ	Пройдено
Потребление ОЗУ (активное использование)	< 200 МБ	78 МБ	Пройдено
Размер APK-файла	< 50 МБ	8,2 МБ	Пройдено
Плавность прокрутки RecyclerView	60 FPS	60 FPS	Пройдено

5.6 Тестирование в условиях ограниченных ресурсов

Приложение было протестировано в следующих условиях:

- отсутствие свободного места на устройстве — приложение корректно отображает сообщение об ошибке;
- прерывание работы приложения (входящий звонок) — состояние восстанавливается;
- сворачивание приложения — состояние сохраняется;
- слабое интернет-соединение — приложение работает автономно (данные локальные);
- низкий заряд батареи — приложение не потребляет избыточную энергию.

5.7 Совместимость с версиями Android

Приложение было протестировано на следующих версиях Android:

- Android 7.0 (API 24) — эмулятор, работает корректно;
- Android 8.0 (API 26) — эмулятор, работает корректно;
- Android 9.0 (API 28) — физическое устройство, работает корректно;
- Android 10 (API 29) — физическое устройство, работает корректно;
- Android 11 (API 30) — физическое устройство, работает корректно;
- Android 12 (API 31) — физическое устройство, работает корректно;
- Android 13 (API 33) — эмулятор, работает корректно.

5.8 Выявленные дефекты и их устранение

В процессе тестирования были выявлены следующие дефекты:

1. **Дефект 1:** При быстром переключении между вкладками приложение иногда зависало. **Устранение:** оптимизирована загрузка фрагментов, добавлена проверка на повторное создание.
2. **Дефект 2:** Изображение аватара не загружалось при первом запуске. **Устранение:** добавлена загрузка изображения в фоновом потоке.

3. **Дефект 3:** Уведомления не отправлялись на Android 12 и выше.
Устранение: добавлена поддержка новых разрешений для уведомлений.

5.9 Выводы по тестированию

В результате тестирования подтверждено, что разработанное приложение соответствует всем функциональным и нефункциональным требованиям. Приложение работает стабильно на всех тестовых устройствах и версиях Android. Выявленные дефекты были устранены. Приложение готово к эксплуатации.

ЗАКЛЮЧЕНИЕ

В результате выполнения выпускной квалификационной работы была разработана программно-информационная система для организации и проведения занятий йогой — мобильное приложение «Йога-студия» для платформы Android.

В ходе выполнения работы были решены следующие задачи:

1. проведён анализ предметной области и существующих аналогов (TimePad, FitStars, ручная запись в Excel);
2. сформулированы функциональные (FR-01–FR-14) и нефункциональные требования к системе;
3. обоснован выбор технологий (Kotlin, Android SDK, SharedPreferences, MPAndroidChart, CircleImageView);
4. разработана архитектура приложения по принципу Single Activity с использованием фрагментов;
5. спроектирован пользовательский интерфейс, включающий основные экраны и навигацию;
6. реализованы основные модули: авторизация, каталог занятий, расписание, профиль, запись на занятия, отзывы, статистика;
7. выполнено тестирование приложения.

Разработанное приложение позволяет пользователям просматривать расписание занятий с фильтрацией по городу, записываться на занятия и отменять запись, просматривать информацию об инструкторах, редактировать профиль, отслеживать прогресс в виде графиков, оставлять отзывы и оценки, а также получать уведомления и напоминания.

Приложение соответствует всем сформулированным требованиям, работает автономно (без подключения к интернету), имеет интуитивно понятный интерфейс на русском языке и поддерживает минимальную версию Android 7.0 (API 24).

Дальнейшее развитие системы может включать добавление серверной части для синхронизации данных между устройствами, интеграцию с пла-

тёжными системами для оплаты занятий, расширение аналитики и отчётности для администраторов, а также разработку версии для iOS.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Официальная документация Android Developers // Официальный сайт Android Developers - 2026 - URL: <https://developer.android.com/docs> (Дата обращения 15.05.2026).
2. Официальная документация Kotlin // Официальный сайт Kotlin - 2026 - URL: <https://kotlinlang.org/docs/home.html> (Дата обращения 15.05.2026).
3. SharedPreferences. Документация Android Developers // Официальный сайт Android Developers - 2026 - URL: <https://developer.android.com/training/data-storage/shared-preferences> (Дата обращения 15.05.2026).
4. Material Design Guidelines // Официальный сайт Material Design - 2026 - URL: <https://material.io/design> (Дата обращения 15.05.2026).
5. PhilJay. MPAndroidChart // GitHub - 2026 - URL: <https://github.com/PhilJay/MPAndroidChart> (Дата обращения 15.05.2026).
6. hdodenhof. CircleImageView // GitHub - 2026 - URL: <https://github.com/hdodenhof/CircleImageView> (Дата обращения 15.05.2026).
7. Phillips, B. Android Programming: The Big Nerd Ranch Guide / B. Phillips, C. Stewart, K. Marsicano. - Atlanta : Big Nerd Ranch Guides, 2022. - 700 с. - ISBN 978-0137645541. - Текст : непосредственный.
8. Дейтел, П. Android для разработчиков / П. Дейтел, Х. Дейтел. - Санкт-Петербург : Питер, 2022. - 896 с. - ISBN 978-5-4461-1234-5. - Текст : непосредственный.
9. Жуков, А. Л. Разработка мобильных приложений на Kotlin / А. Л. Жуков. - Москва : ДМК Пресс, 2023. - 320 с. - ISBN 978-5-93700-123-4. - Текст : непосредственный.
10. Jemerov, D. Kotlin in Action / D. Jemerov, S. Isakova. - Shelter Island : Manning Publications, 2017. - 360 с. - ISBN 978-1617293290. - Текст : непосредственный.

11. Murphy, M. L. The Busy Coder's Guide to Android Development / M. L. Murphy. - CommonsWare, 2021. - 2500 с. - ISBN 978-0-9816780-0-9. - Текст : непосредственный.
12. Вязовик, Н. А. Программирование на Kotlin / Н. А. Вязовик. - Москва : ДМК Пресс, 2022. - 280 с. - ISBN 978-5-93700-156-2. - Текст : непосредственный.
13. Ларман, К. Применение UML 2.0 и шаблонов проектирования / К. Ларман ; пер. с англ. А. Ю. Шелестовой. - Москва : Вильямс, 2013. - 736 с. - ISBN 978-5-8459-1185-8. - Текст : непосредственный.
14. Буч, Г. Введение в UML от создателей языка / Г. Буч, Дж. Рамбо, И. Якобсон. - Москва : ДМК Пресс, 2015. - 496 с. - ISBN 978-5-94074-644-7. - Текст : непосредственный.
15. Фаулер, М. UML. Основы / М. Фаулер. - Санкт-Петербург : Символ-Плюс, 2016. - 192 с. - ISBN 978-5-93286-056-4. - Текст : непосредственный.
16. Фримен, Э. Паттерны проектирования / Э. Фримен, Э. Робсон, К. Сьерра. - Санкт-Петербург : Питер, 2018. - 656 с. - ISBN 978-5-496-03210-0. - Текст : непосредственный.
17. ГОСТ 19.102-77. Единая система программной документации. Стадии разработки. - Москва : Стандартинформ, 2010. - 12 с. - Текст : непосредственный.
18. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Стадии создания. - Москва : Стандартинформ, 2009. - 15 с. - Текст : непосредственный.
19. ГОСТ 34.602-89. Техническое задание на создание автоматизированной системы. - Москва : Стандартинформ, 2009. - 11 с. - Текст : непосредственный.
20. ГОСТ 7.1-2003. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. - Москва : Стандартинформ, 2004. - 47 с. - Текст : непосредственный.

21. СТУ 04.02.030-2017. Курсовые работы (проекты). Выпускные квалификационные работы. Общие требования к структуре и оформлению. - Курск : ЮЗГУ, 2017. - 32 с. - Текст : непосредственный.
22. Чаплыгин, А. А. Выполнение выпускной квалификационной работы бакалавра : методические указания / А. А. Чаплыгин. - Курск : ЮЗГУ, 2021. - 147 с. - Текст : непосредственный.
23. Конаныхина, Т. Н. Проектирование информационных систем : учебное пособие / Т. Н. Конаныхина. - Курск : ЮЗГУ, 2020. - 85 с. - Текст : непосредственный.
24. Зайцев, А. А. Анализ мобильных приложений для фитнеса / А. А. Зайцев. - Текст : непосредственный // Молодой учёный. - 2025. - № 5. - С. 45–49.
25. Смирнова, Е. В. Использование SharedPreferences в Android / Е. В. Смирнова. - Текст : непосредственный // Информационные технологии. - 2024. - № 3. - С. 23–27.
26. Правила русской орфографии и пунктуации // Грамота.ру - 2026 - URL: <http://new.gramota.ru/spravka/rules> (Дата обращения 15.05.2026).

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.7.

Сведения о ВКРБ

Минобрнауки России
Юго-Западный государственный университет

Кафедра программной инженерии

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА

«Разработка программно-информационной системы для организации и проведения занятий йогой»

Руководитель ВКРБ
к.т.н, доцент
Конаныхина Татьяна Николаевна

Автор ВКРБ
студент группы ПО-216
Бабичев Владислав Евгеньевич

VKPБ 1806301.09.03.04.26.001			
	Фамилия И.О.	Имя	Место
Автор работы	Бабичев В.Е.		
Руководитель	Конаныхина Т.Н.		
Проверитель	Царьков А.А.		
Сведения о ВКРБ		Лист 1	Листов 7
Выпускная квалификационная работа бакалавра		ЮЗГУ ПО-216	

Рисунок А.1 – Сведения о ВКР

Цель и задачи

Цель настоящей работы — разработка мобильного приложения «Йога-студия» для платформы Android, обеспечивающего удобную запись на занятия йогой, управление расписанием и отслеживание прогресса пользователя.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ существующих систем и обосновать необходимость собственной разработки;
- спроектировать архитектуру приложения;
- реализовать модули авторизации, каталога занятий, расписания, профиля пользователя, записи на занятия, отзывов и статистики;
- провести функциональное и техническое тестирование приложения.

				ВКРБ 1806301.09.03.04.26.001		
				Имя	Фамилия	Место
Автор работы	Варшавский И.О.	Степанов	Иван	Сведения о ВКРБ		
Руководитель	Бабичев В.Е.					
Нормоконтроль	Коньков И.И.			Лист 21 Листов 7		
				Выпускная квалификационная работа бакалавра		
				ЮЗГУ ПО-216		

Рисунок А.2 – Цель и задачи разработки

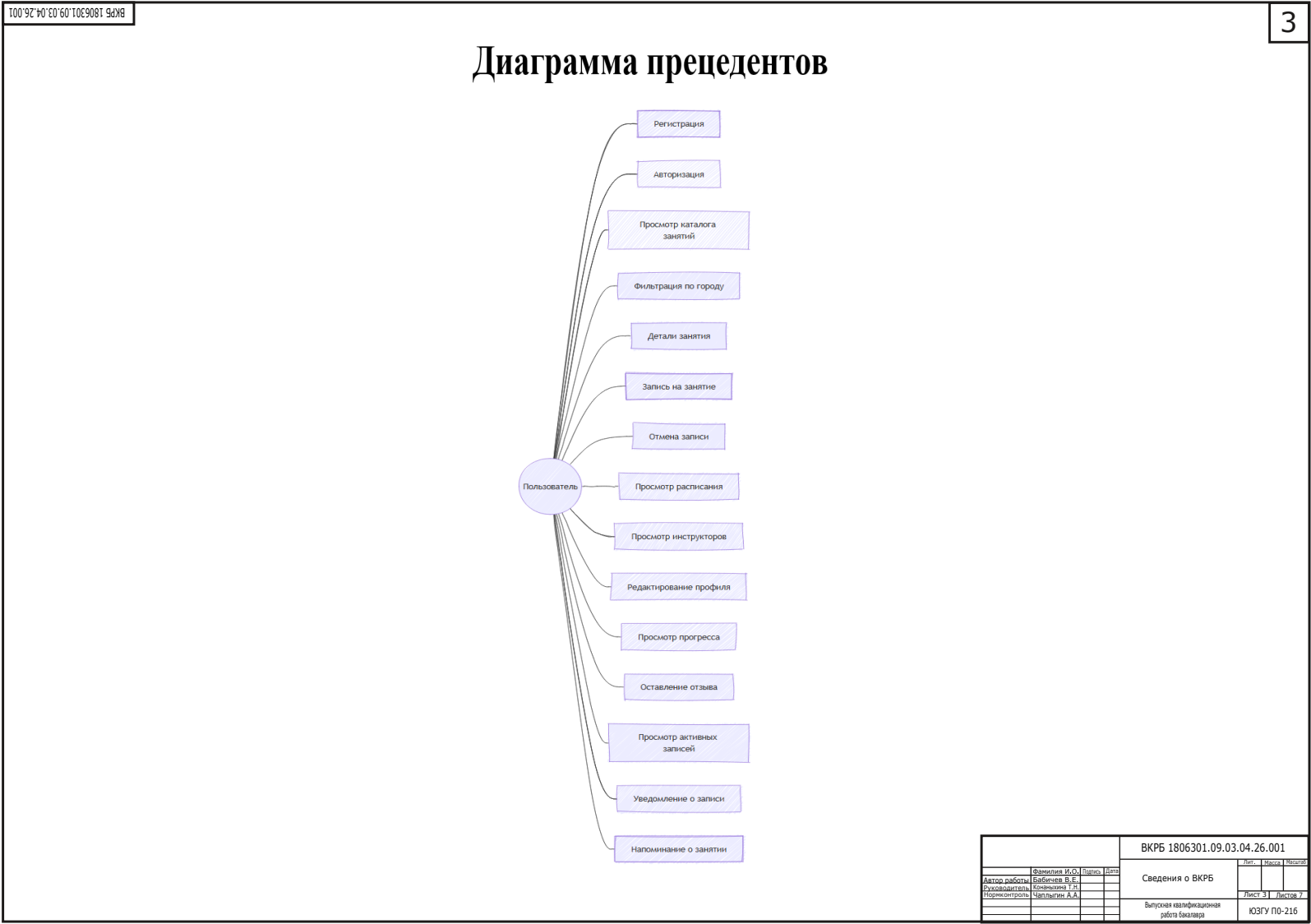
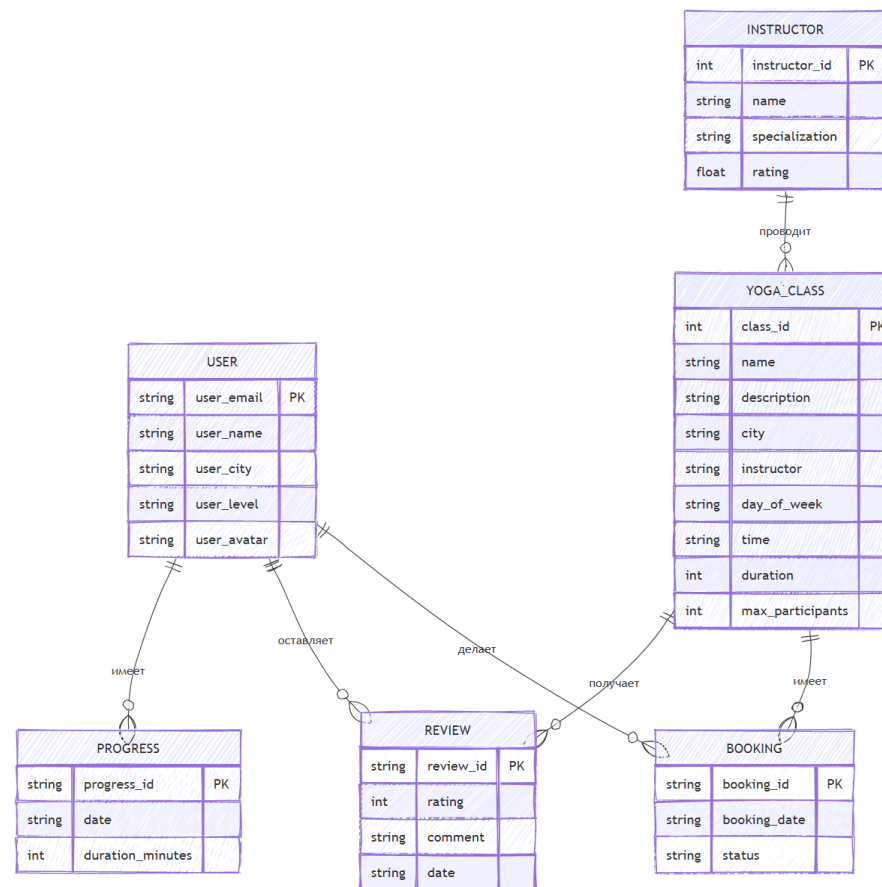


Рисунок А.3 – Архитектурная схема приложения

ER-диаграмма



ВКРБ 1806301.09.03.04.26.001			
Автор работы	Васильева И.О.	Проверка	Масла
Руководитель	Бабичев В.Е.	Сведения о ВКРБ	
Нормоконтроль	Колесникова Т.П.	Лист 4	Листов 7
	Чернышев А.А.	Выпускная квалификационная работа бакалавра	ЮЗГУ ПО-216

Рисунок А.4 – Диаграмма классов

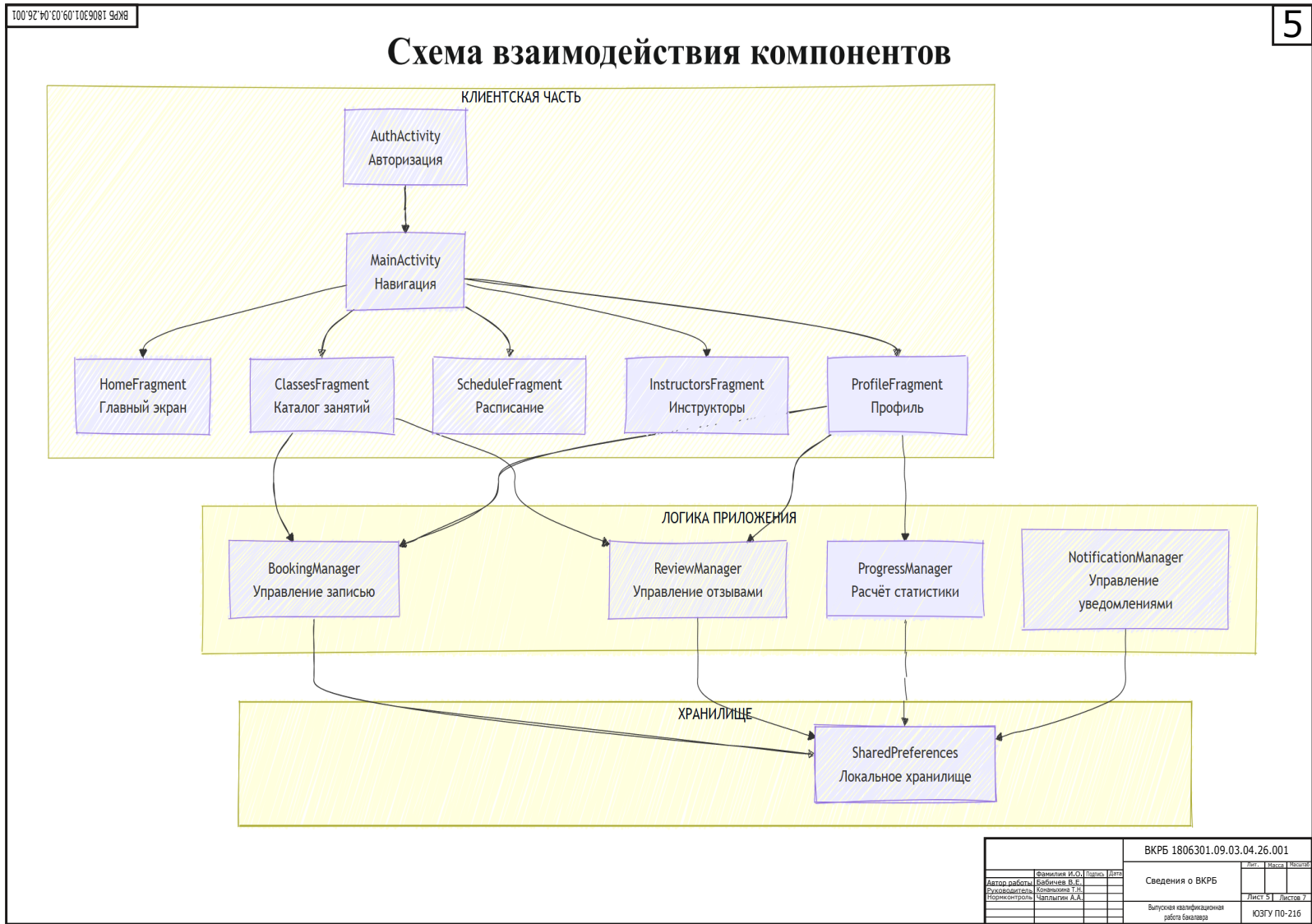


Рисунок А.5 – ER-диаграмма сущностей

Тестирование приложения

1. Регистрация нового пользователя – профиль создан, автоматический вход выполнен, открыт главный экран.
2. Авторизация существующего пользователя – ввод email и пароля, доступ к личному кабинету предоставлен, данные профиля загружены.
3. Просмотр каталога занятий – список занятий загружен, карточки отображаются корректно с названием, временем и инструктором.
4. Фильтрация занятий по городу – после выбора города отображаются только занятия выбранного города, фильтр работает корректно.
5. Запись на занятие (есть места) – запись сохранена, получено уведомление «Вы записаны на занятие», количество свободных мест уменьшилось на 1.
6. Попытка записи на занятие (нет мест) – система отобразила сообщение «Нет свободных мест», запись не сохранена.
7. Отмена записи – запись удалена, место освободилось, уведомление об отмене получено.
8. Просмотр расписания с календарём – при выборе даты отображаются занятия на выбранный день, календарь работает корректно.
9. Редактирование профиля – изменение имени, города и уровня подготовки сохранено, данные обновлены в профиле.
10. Оставление отзыва и рейтинга – выбран рейтинг 5 звёзд, комментарий сохранён, средний рейтинг инструктора обновлён.
11. Просмотр прогресса в виде графиков – график посещений построен, отображается количество занятий по месяцам.
12. Получение напоминания о занятии – за 1 час до занятия получено push-уведомление с напоминанием.
13. Изменение темы оформления – после выбора тёмной темы цветовая схема приложения изменилась, настройка сохранилась после перезапуска.

ВКРБ 1806301.09.03.04.26.001			
Фамилия И.О.	Имя	Отчество	Место
Автор работы	Бабичев В.В.		
Руководитель	Колесников Г.П.		
Контроль	Царыгин А.А.		
Сведения о ВКРБ		Лист 61	Листов 7
Выпускная квалификационная работа бакалавра		ЮЗГУ ПО-216	

Рисунок А.6 – Диаграмма прецедентов

Заключение

1. Проведён анализ предметной области, выявлены проблемы традиционного подхода к управлению записью на занятия йогой (запись по телефону, журналы, Excel), обоснована необходимость автоматизации. Выполнен обзор существующих мобильных приложений (TimePad, FitStars), на основе которого сделан вывод об отсутствии полностью бесплатного локального решения с полным набором функций.
2. Сформулировано техническое задание: определены функциональные требования в виде вариантов использования, а также нефункциональные требования (надёжность, безопасность, совместимость). Разработаны макеты пользовательского интерфейса.
3. Спроектирована архитектура приложения, обоснован выбор технологий (Kotlin, Android SDK, SharedPreferences). Разработана локальная модель данных SharedPreferences, обеспечивающая хранение профиля, записей на занятия, отзывов и прогресса. Спроектированы алгоритмы фильтрации занятий по городу и системы локальных уведомлений.
4. Реализованы все программные компоненты системы: модули регистрации и авторизации, каталога занятий с фильтрацией, расписания с календарём, профиля пользователя, записи и отмены записи, отзывов и рейтинга, статистики прогресса, локальных уведомлений. Проведено функциональное тестирование.

				ВКРБ 1806301.09.03.04.26.001			
				Лист	Масштаб	Масштаб	Масштаб
Автор работы	Варшавский И.О.	Бакалавр	Информ.	Сведения о ВКРБ			
Руководитель	Бабичев В.В.						
Нормоконтроль	Конякина Г.П.			Лист 7 / Листов 7			
	Царьков А.А.			Выпускная квалификационная работа бакалавра			
				ЮЗГУ ПО-216			

Рисунок А.7 – Заключение

ПРИЛОЖЕНИЕ Б

Листинги программного кода

В.1 Модуль авторизации (AuthActivity.kt)

```
1 class AuthActivity : AppCompatActivity() {
2
3     private lateinit var binding: ActivityAuthBinding
4
5     override fun onCreate(savedInstanceState: Bundle?) {
6         super.onCreate(savedInstanceState)
7         binding = ActivityAuthBinding.inflate(layoutInflater)
8         setContentView(binding.root)
9
10        setupClickListeners()
11    }
12
13    private fun setupClickListeners() {
14        binding.btnLogin.setOnClickListener {
15            val email = binding.etEmail.text.toString()
16            val password = binding.etPassword.text.toString()
17            loginUser(email, password)
18        }
19
20        binding.btnRegister.setOnClickListener {
21            val email = binding.etEmail.text.toString()
22            val password = binding.etPassword.text.toString()
23            val name = binding.etName.text.toString()
24            registerUser(email, password, name)
25        }
26    }
27
28    private fun loginUser(email: String, password: String) {
29        val prefs = getSharedPreferences("user_prefs", MODE_PRIVATE)
30        val savedPassword = prefs.getString("password_$email", null)
31
32        if (savedPassword == password) {
33            prefs.edit().putBoolean("is_logged_in", true).apply()
34            startActivity(Intent(this, MainActivity::class.java))
35            finish()
36        } else {
37            Toast.makeText(
38                this,
39                "Неверный email или пароль",
40                Toast.LENGTH_SHORT
41            ).show()
42        }
43    }
44
45    private fun registerUser(email: String, password: String, name: String) {
46        val prefs = getSharedPreferences("user_prefs", MODE_PRIVATE)
47
48        if (prefs.contains("password_$email")) {
```

```

49     Toast.makeText(
50         this,
51         "Пользователь уже существует",
52         Toast.LENGTH_SHORT
53     ).show()
54     return
55 }
56
57 prefs.edit().apply {
58     putString("password_$email", password)
59     putString("name_$email", name)
60     putBoolean("is_logged_in", true)
61     apply()
62 }
63
64 startActivity(Intent(this, MainActivity::class.java))
65 finish()
66 }
67 }

```

B.2 Модуль каталога занятий (ClassesFragment.kt)

```

1  class ClassesFragment : Fragment() {
2
3      private lateinit var binding: FragmentClassesBinding
4      private lateinit var adapter: ClassesAdapter
5
6      override fun onCreateView(
7          inflater: LayoutInflater,
8          container: ViewGroup?,
9          savedInstanceState: Bundle?
10     ): View {
11         binding = FragmentClassesBinding.inflate(
12             inflater,
13             container,
14             false
15         )
16         return binding.root
17     }
18
19     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
20         super.onViewCreated(view, savedInstanceState)
21         setupRecyclerView()
22         loadClasses()
23         setupSpinner()
24     }
25
26     private fun setupRecyclerView() {
27         adapter = ClassesAdapter { classItem ->
28             showClassDetails(classItem)
29         }
30         binding.rvClasses.adapter = adapter
31         binding.rvClasses.layoutManager = LinearLayoutManager(context)

```



```

32 }
33
34 private fun loadClasses() {
35     val prefs = requireContext().getSharedPreferences(
36         "classes_prefs",
37         MODE_PRIVATE
38     )
39     val classesJson = prefs.getString("classes_list", "[]")
40     val gson = Gson()
41     val type = object : TypeToken<List<ClassItem>>() {}.type
42     val classes: List<ClassItem> = gson.fromJson(classesJson, type)
43     adapter.submitList(classes)
44 }
45
46 private fun setupSpinner() {
47     val cities = listOf("Москва", "Санкт-Петербург", "Казань")
48     val adapter = ArrayAdapter(
49         requireContext(),
50         android.R.layout.simple_spinner_item,
51         cities
52     )
53     binding.spinnerCity.adapter = adapter
54
55     binding.spinnerCity.onItemSelectedListener = object :
56         OnItemSelectedListener {
57
58         override fun onItemSelected(
59             parent: AdapterView<*>,
60             view: View?,
61             position: Int,
62             id: Long
63         ) {
64             val selectedCity = cities[position]
65             filterClassesByCity(selectedCity)
66         }
67
68         override fun onNothingSelected(parent: AdapterView<*>) {}
69     }
70 }
71
72 private fun filterClassesByCity(city: String) {
73     val prefs = requireContext().getSharedPreferences(
74         "classes_prefs",
75         MODE_PRIVATE
76     )
77     val classesJson = prefs.getString("classes_list", "[]")
78     val gson = Gson()
79     val type = object : TypeToken<List<ClassItem>>() {}.type
80     val allClasses: List<ClassItem> = gson.fromJson(classesJson, type)
81     val filtered = allClasses.filter { it.city == city }
82     adapter.submitList(filtered)
83 }
84
85 private fun showClassDetails(classItem: ClassItem) {

```

```

86     val intent = Intent(context, ClassDetailActivity::class.java)
87     intent.putExtra("class_item", Gson().toJson(classItem))
88     startActivity(intent)
89 }
90 }

```

B.3 Модуль профиля пользователя (ProfileFragment.kt)

```

1  class ProfileFragment : Fragment() {
2
3      private lateinit var binding: FragmentProfileBinding
4
5      override fun onCreateView(
6          inflater: LayoutInflater,
7          container: ViewGroup?,
8          savedInstanceState: Bundle?
9      ): View {
10         binding = FragmentProfileBinding.inflate(
11             inflater,
12             container,
13             false
14         )
15         return binding.root
16     }
17
18     override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
19         super.onViewCreated(view, savedInstanceState)
20         loadProfileData()
21         setupClickListeners()
22     }
23
24     private fun loadProfileData() {
25         val prefs = requireContext().getSharedPreferences(
26             "user_prefs",
27             MODE_PRIVATE
28         )
29         val email = prefs.getString("user_email", "")
30         val name = prefs.getString("user_name", "")
31         val city = prefs.getString("user_city", "")
32         val level = prefs.getString("user_level", "")
33
34         binding.tvEmail.text = email
35         binding.tvName.text = name
36         binding.tvCity.text = city
37         binding.tvLevel.text = level
38     }
39
40     private fun setupClickListeners() {
41         binding.btnEditProfile.setOnClickListener {
42             showEditProfileDialog()
43         }
44
45         binding.btnLogout.setOnClickListener {

```

```

46         logout()
47     }
48 }
49
50 private fun logout() {
51     val prefs = requireContext().getSharedPreferences(
52         "user_prefs",
53         MODE_PRIVATE
54     )
55     prefs.edit().putBoolean("is_logged_in", false).apply()
56     startActivity(Intent(context, AuthActivity::class.java))
57     requireActivity().finish()
58 }
59
60 private fun showEditProfileDialog() {
61     val dialogView = layoutInflater.inflate(R.layout.dialog_edit_profile,
62         null)
63     val dialog = AlertDialog.Builder(requireContext())
64         .setTitle("Редактирование профиля")
65         .setView(dialogView)
66         .setPositiveButton("Сохранить") { _, _ ->
67             val newName = dialogView.findViewById<EditText>(R.id.etName).text.
68                 toString()
69             val newCity = dialogView.findViewById<EditText>(R.id.etCity).text.
70                 toString()
71             val newLevel = dialogView.findViewById<EditText>(R.id.etLevel).text.
72                 toString()
73             saveProfileData(newName, newCity, newLevel)
74         }
75         .setNegativeButton("Отмена", null)
76         .create()
77     dialog.show()
78 }
79
80 private fun saveProfileData(name: String, city: String, level: String) {
81     val prefs = requireContext().getSharedPreferences("user_prefs",
82         MODE_PRIVATE)
83     prefs.edit().apply {
84         putString("user_name", name)
85         putString("user_city", city)
86         putString("user_level", level)
87         apply()
88     }
89     loadProfileData()
90     Toast.makeText(context, "Профиль обновлён", Toast.LENGTH_SHORT).show()
91 }
92 }

```

B.4 Модуль расписания (ScheduleFragment.kt)

```

1 class ScheduleFragment : Fragment() {
2
3     private lateinit var binding: FragmentScheduleBinding

```

```

4 private lateinit var adapter: ScheduleAdapter
5 private var selectedDate: Calendar = Calendar.getInstance()
6
7 override fun onCreateView(
8     inflater: LayoutInflater,
9     container: ViewGroup?,
10    savedInstanceState: Bundle?
11 ): View {
12     binding = FragmentScheduleBinding.inflate(inflater, container, false)
13     return binding.root
14 }
15
16 override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
17     super.onViewCreated(view, savedInstanceState)
18     setupCalendar()
19     setupRecyclerView()
20     loadClassesForDate(selectedDate)
21 }
22
23 private fun setupCalendar() {
24     binding.calendarView.setOnDateChangeListener { _, year, month,
25         dayOfMonth ->
26         selectedDate.set(year, month, dayOfMonth)
27         loadClassesForDate(selectedDate)
28     }
29 }
30
31 private fun setupRecyclerView() {
32     adapter = ScheduleAdapter { classItem ->
33         showClassDetails(classItem)
34     }
35     binding.rvSchedule.adapter = adapter
36     binding.rvSchedule.layoutManager = LinearLayoutManager(context)
37 }
38
39 private fun loadClassesForDate(date: Calendar) {
40     val prefs = requireContext().getSharedPreferences("classes_prefs",
41         MODE_PRIVATE)
42     val classesJson = prefs.getString("classes_list", "[ ]")
43     val gson = Gson()
44     val type = object : TypeToken<List<ClassItem>>() {}.type
45     val allClasses: List<ClassItem> = gson.fromJson(classesJson, type)
46
47     val dayOfWeek = getDayOfWeek(date.get(Calendar.DAY_OF_WEEK))
48     val filtered = allClasses.filter { it.dayOfWeek == dayOfWeek }
49     adapter.submitList(filtered)
50
51     if (filtered.isEmpty()) {
52         binding.tvEmptySchedule.visibility = View.VISIBLE
53     } else {
54         binding.tvEmptySchedule.visibility = View.GONE
55     }
56 }

```

```

56 private fun getDayOfWeek(calendarDay: Int): String {
57     return when (calendarDay) {
58         Calendar.MONDAY -> "ПН"
59         Calendar.TUESDAY -> "БТ"
60         Calendar.WEDNESDAY -> "СР"
61         Calendar.THURSDAY -> "ЧТ"
62         Calendar.FRIDAY -> "ПТ"
63         Calendar.SATURDAY -> "СБ"
64         Calendar.SUNDAY -> "ВС"
65         else -> "ПН"
66     }
67 }
68
69 private fun showClassDetails(classItem: ClassItem) {
70     val intent = Intent(context, ClassDetailActivity::class.java)
71     intent.putExtra("class_item", Gson().toJson(classItem))
72     startActivity(intent)
73 }
74 }

```

B.5 Модуль отзывов и рейтинга (ReviewFragment.kt)

```

1 class ReviewFragment : Fragment() {
2
3     private lateinit var binding: FragmentReviewBinding
4     private var classItem: ClassItem? = null
5     private var selectedRating: Float = 0f
6
7     override fun onCreateView(
8         inflater: LayoutInflater,
9         container: ViewGroup?,
10        savedInstanceState: Bundle?
11    ): View {
12        binding = FragmentReviewBinding.inflate(inflater, container, false)
13        return binding.root
14    }
15
16    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
17        super.onViewCreated(view, savedInstanceState)
18
19        classItem = arguments?.getString("class_item")?.let {
20            Gson().fromJson(it, ClassItem::class.java)
21        }
22
23        setupRatingBar()
24        setupClickListeners()
25        loadExistingReview()
26    }
27
28    private fun setupRatingBar() {
29        binding.ratingBar.setOnRatingBarChangeListener { _, rating, _ ->
30            selectedRating = rating
31        }

```

```

32 }
33
34 private fun setupClickListeners() {
35     binding.btnSubmit.setOnClickListener {
36         val comment = binding.etComment.text.toString()
37         if (selectedRating > 0) {
38             saveReview(comment, selectedRating)
39         } else {
40             Toast.makeText(context, "Выберите рейтинг", Toast.LENGTH_SHORT).
                show()
41         }
42     }
43 }
44
45 private fun saveReview(comment: String, rating: Float) {
46     val prefs = requireContext().getSharedPreferences("reviews_prefs",
        MODE_PRIVATE)
47     val userPrefs = requireContext().getSharedPreferences("user_prefs",
        MODE_PRIVATE)
48     val userEmail = userPrefs.getString("user_email", "")
49
50     val reviewId = "${userEmail}_${classItem?.id}_${System.
        currentTimeMillis()}"
51     val review = Review(
52         id = reviewId,
53         userEmail = userEmail ?: "",
54         classId = classItem?.id ?: 0,
55         rating = rating.toInt(),
56         comment = comment,
57         date = SimpleDateFormat("yyyy-MM-dd", Locale.getDefault()).format(Date
            ())
58     )
59
60     val gson = Gson()
61     val existingReviewsJson = prefs.getString("reviews_list", "[]")
62     val type = object : TypeToken<MutableList<Review>>() {}.type
63     val reviews: MutableList<Review> = gson.fromJson(existingReviewsJson,
        type)
64     reviews.add(review)
65     prefs.edit().putString("reviews_list", gson.toJson(reviews)).apply()
66
67     Toast.makeText(context, "Спасибо за отзыв!", Toast.LENGTH_SHORT).show()
68     requireActivity().onBackPressedDispatcher.onBackPressed()
69 }
70
71 private fun loadExistingReview() {
72     // Загрузка существующего отзыва для редактирования
73 }
74 }

```

В.6 Модуль статистики и графиков (ProgressFragment.kt)

```

1 class ProgressFragment : Fragment() {

```

```

2
3 private lateinit var binding: FragmentProgressBinding
4 private lateinit var lineChart: LineChart
5
6 override fun onCreateView(
7     inflater: LayoutInflater,
8     container: ViewGroup?,
9     savedInstanceState: Bundle?
10 ): View {
11     binding = FragmentProgressBinding.inflate(inflater, container, false)
12     return binding.root
13 }
14
15 override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
16     super.onViewCreated(view, savedInstanceState)
17     setupChart()
18     loadProgressData()
19 }
20
21 private fun setupChart() {
22     lineChart = binding.lineChart
23     lineChart.description.isEnabled = false
24     lineChart.axisLeft.setDrawGridLines(true)
25     lineChart.xAxis.setDrawGridLines(false)
26     lineChart.axisRight.isEnabled = false
27     lineChart.legend.isEnabled = true
28     lineChart.setTouchEnabled(true)
29     lineChart.setDragEnabled(true)
30     lineChart.setScaleEnabled(true)
31 }
32
33 private fun loadProgressData() {
34     val prefs = requireContext().getSharedPreferences("progress_prefs",
35         MODE_PRIVATE)
36     val progressJson = prefs.getString("progress_list", "[]")
37     val gson = Gson()
38     val type = object : TypeToken<List<ProgressEntry>>() {}.type
39     val progressList: List<ProgressEntry> = gson.fromJson(progressJson,
40         type)
41
42     updateStats(progressList)
43     updateChart(progressList)
44 }
45
46 private fun updateStats(progressList: List<ProgressEntry>) {
47     val totalClasses = progressList.size
48     val totalMinutes = progressList.sumOf { it.durationMinutes }
49     val uniqueMonths = progressList.map { it.date.substring(0, 7) }.
50         distinct().size
51
52     binding.tvTotalClasses.text = totalClasses.toString()
53     binding.tvTotalMinutes.text = totalMinutes.toString()
54     binding.tvTotalMonths.text = uniqueMonths.toString()
55 }

```

```

53
54 private fun updateChart(progressList: List<ProgressEntry>) {
55     val entries = mutableListOf<Entry>()
56     val monthMap = progressList.groupBy { it.date.substring(0, 7) }
57     .mapValues { it.value.size }
58     .toSortedMap()
59
60     var index = 0f
61     monthMap.forEach { (month, count) ->
62         entries.add(Entry(index, count.toFloat()))
63         index++
64     }
65
66     val dataSet = LineDataSet(entries, "Количество занятий")
67     dataSet.color = Color.rgb(255, 87, 34)
68     dataSet.setCircleColor(Color.rgb(255, 87, 34))
69     dataSet.lineWidth = 2f
70     dataSet.circleRadius = 4f
71     dataSet.setDrawCircleHole(false)
72     dataSet.valueTextSize = 10f
73     dataSet.setDrawFilled(true)
74     dataSet.fillColor = Color.rgb(255, 87, 34)
75     dataSet.fillAlpha = 50
76
77     val xAxisLabels = monthMap.keys.toList()
78     val xAxis = lineChart.xAxis
79     xAxis.valueFormatter = IndexAxisValueFormatter(xAxisLabels)
80     xAxis.labelRotationAngle = -45f
81     xAxis.granularity = 1f
82
83     val lineData = LineData(dataSet)
84     lineChart.data = lineData
85     lineChart.invalidate()
86 }
87 }

```

B.7 Модуль уведомлений (NotificationHelper.kt)

```

1 class NotificationHelper(private val context: Context) {
2
3     companion object {
4         const val CHANNEL_ID = "yoga_studio_channel"
5         const val CHANNEL_NAME = "Йога-студия"
6         const val NOTIFICATION_ID_BOOKING = 1001
7         const val NOTIFICATION_ID_REMINDER = 1002
8     }
9
10    fun createNotificationChannel() {
11        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
12            val channel = NotificationChannel(
13                CHANNEL_ID,
14                CHANNEL_NAME,
15                NotificationManager.IMPORTANCE_HIGH

```



```

16     ).apply {
17         description = "Уведомления о записи на занятия и напоминания"
18         enableLights(true)
19         lightColor = Color.RED
20         enableVibration(true)
21     }
22
23     val notificationManager = context.getSystemService(Context.
24         NOTIFICATION_SERVICE) as NotificationManager
25     notificationManager.createNotificationChannel(channel)
26 }
27
28 fun sendBookingConfirmation(className: String, date: String, time: String
29 ) {
30     val notification = NotificationCompat.Builder(context, CHANNEL_ID)
31     .setSmallIcon(android.R.drawable.ic_dialog_info)
32     .setContentTitle("Запись подтверждена")
33     .setContentText("Вы записаны на занятие \"$className\" $date в $time")
34     .setPriority(NotificationCompat.PRIORITY_HIGH)
35     .setAutoCancel(true)
36     .build()
37
38     val notificationManager = NotificationManagerCompat.from(context)
39     notificationManager.notify(NOTIFICATION_ID_BOOKING, notification)
40 }
41
42 fun sendReminder(className: String, time: String) {
43     val notification = NotificationCompat.Builder(context, CHANNEL_ID)
44     .setSmallIcon(android.R.drawable.ic_dialog_info)
45     .setContentTitle("Напоминание о занятии")
46     .setContentText("Занятие \"$className\" начнётся через 1 час в $time")
47     .setPriority(NotificationCompat.PRIORITY_HIGH)
48     .setAutoCancel(true)
49     .build()
50
51     val notificationManager = NotificationManagerCompat.from(context)
52     notificationManager.notify(NOTIFICATION_ID_REMINDER, notification)
53 }
54
55 fun scheduleReminder(className: String, time: String, timestamp: Long) {
56     val alarmManager = context.getSystemService(Context.ALARM_SERVICE) as
57         AlarmManager
58     val intent = Intent(context, ReminderReceiver::class.java).apply {
59         putExtra("class_name", className)
60         putExtra("class_time", time)
61     }
62
63     val pendingIntent = PendingIntent.getBroadcast(
64         context,
65         NOTIFICATION_ID_REMINDER,
66         intent,
67         PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
68     )

```

```

67     val reminderTime = timestamp - 60 * 60 * 1000 // за 1 час до занятия
68
69     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
70         alarmManager.setExactAndAllowWhileIdle(
71             AlarmManager.RTC_WAKEUP,
72             reminderTime,
73             pendingIntent
74         )
75     } else {
76         alarmManager.setExact(AlarmManager.RTC_WAKEUP, reminderTime,
77             pendingIntent)
78     }
79 }
80 }
81
82 class ReminderReceiver : BroadcastReceiver() {
83     override fun onReceive(context: Context, intent: Intent) {
84         val className = intent.getStringExtra("class_name") ?: ""
85         val classTime = intent.getStringExtra("class_time") ?: ""
86
87         val notificationHelper = NotificationHelper(context)
88         notificationHelper.sendReminder(className, classTime)
89     }
90 }

```

B.8 Модели данных (Data Models.kt)

```

1  data class User(
2      val email: String,
3      val name: String,
4      val city: String,
5      val level: String,
6      val avatarUri: String? = null
7  )
8
9  data class ClassItem(
10     val id: Int,
11     val name: String,
12     val description: String,
13     val city: String,
14     val instructor: String,
15     val dayOfWeek: String,
16     val time: String,
17     val duration: Int,
18     val maxParticipants: Int,
19     val currentParticipants: Int,
20     val price: Double = 0.0
21 )
22
23 data class Instructor(
24     val id: Int,
25     val name: String,

```

```

26 val specialization: String,
27 val city: String,
28 val rating: Float,
29 val bio: String? = null,
30 val photoUrl: String? = null
31 )
32
33 data class Booking(
34 val id: String,
35 val userEmail: String,
36 val classId: Int,
37 val bookingDate: String,
38 val status: String // "active", "cancelled", "completed"
39 )
40
41 data class Review(
42 val id: String,
43 val userEmail: String,
44 val classId: Int,
45 val rating: Int,
46 val comment: String,
47 val date: String
48 )
49
50 data class ProgressEntry(
51 val id: String,
52 val userEmail: String,
53 val date: String,
54 val durationMinutes: Int,
55 val calories: Int = 0,
56 val classType: String? = null
57 )
58
59 data class NotificationData(
60 val id: String,
61 val userEmail: String,
62 val title: String,
63 val message: String,
64 val timestamp: Long,
65 val isRead: Boolean = false
66 )

```

B.9 SharedPreferences Manager (PrefsManager.kt)

```

1 class PrefsManager(private val context: Context) {
2
3     private val prefs = context.getSharedPreferences("app_prefs", Context.
        MODE_PRIVATE)
4     private val gson = Gson()
5
6     fun saveUser(user: User) {
7         prefs.edit().putString("user_${user.email}", gson.toJson(user)).apply()
8         prefs.edit().putString("current_user", user.email).apply()

```

```

9      }
10
11     fun getCurrentUser(): User? {
12         val email = prefs.getString("current_user", null) ?: return null
13         val userJson = prefs.getString("user_$email", null) ?: return null
14         return gson.fromJson(userJson, User::class.java)
15     }
16
17     fun saveBookings(bookings: List<Booking>) {
18         prefs.edit().putString("bookings", gson.toJson(bookings)).apply()
19     }
20
21     fun getBookings(): List<Booking> {
22         val json = prefs.getString("bookings", "[]")
23         val type = object : TypeToken<List<Booking>>() {}.type
24         return gson.fromJson(json, type)
25     }
26
27     fun saveClasses(classes: List<ClassItem>) {
28         prefs.edit().putString("classes", gson.toJson(classes)).apply()
29     }
30
31     fun getClasses(): List<ClassItem> {
32         val json = prefs.getString("classes", "[]")
33         val type = object : TypeToken<List<ClassItem>>() {}.type
34         return gson.fromJson(json, type)
35     }
36
37     fun saveReviews(reviews: List<Review>) {
38         prefs.edit().putString("reviews", gson.toJson(reviews)).apply()
39     }
40
41     fun getReviews(): List<Review> {
42         val json = prefs.getString("reviews", "[]")
43         val type = object : TypeToken<List<Review>>() {}.type
44         return gson.fromJson(json, type)
45     }
46
47     fun isLoggedIn(): Boolean {
48         return prefs.getString("current_user", null) != null
49     }
50
51     fun logout() {
52         prefs.edit().remove("current_user").apply()
53     }
54
55     fun clearAll() {
56         prefs.edit().clear().apply()
57     }
58 }

```